Learnable Hodge–Laplacian Driven Mixture of Experts for Steady-PDE Surrogates on Coarse Unstructured Meshes

Stavros Nousias[1], **Akis Nousias**[2], **Konstantinos Gkrispanis**[1]

[1]Technical University of Munich
[2]K3Y Labs

Abstract

Surrogate models for steady partial differential equations on unstructured meshes often apply a single mixing mechanism to the whole solution. We propose a mixture of experts architecture supervised at different spectral scales. A controller that utilizes learnable laplacian matrices, referred to as Hodge Controller, receives the mesh geometry and input features and produces per-vertex weights that combine the expert representations. During training, a separate discrete Laplacian $L_0 = \star_0^{-1} d_0^\top \star_1 d_0$ decomposes the reference fields into scale-dependent targets for auxiliary expert heads. We evaluate the model on AirFRANS, a steady Reynolds-averaged Navier–Stokes (RANS) airfoil benchmark, using remeshed triangular grids with approximately 9.6k faces and 4.9k vertices. On the official $800/200$ split, our model achieves 31% lower relative L_2 error and 47% lower interior mean squared error than Transolver on the same evaluation meshes, with approximately half as many parameters. Training allocations and checkpoint-selection procedures differ between our model and the baselines. Code: https://github.com/snousias/hodge-moe-pde.

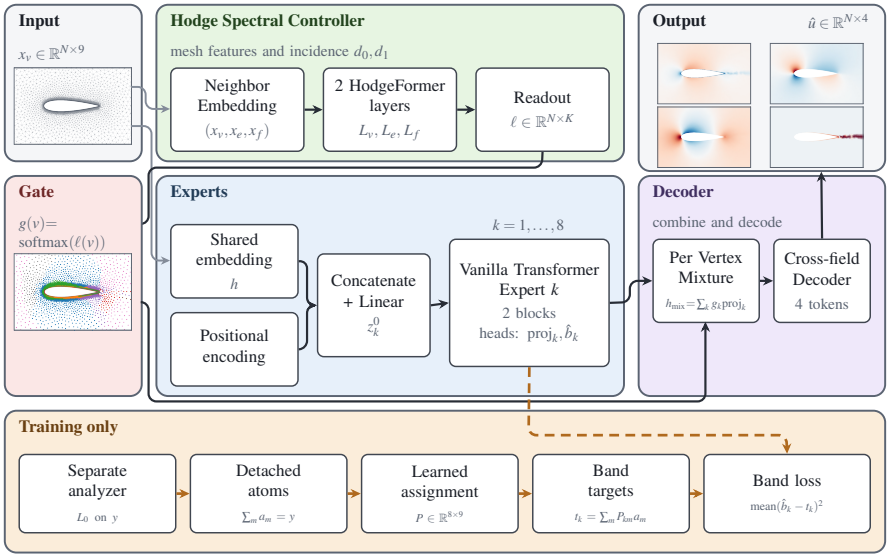


Figure 1: Overview of the model. The Hodge controller routes transformer experts that combine shared vertex embeddings with positional encodings at different frequency scales. Their representations are mixed and decoded into four fields. During training, a separate analyzer decomposes the reference fields into atoms, which a learned assignment combines into targets for the auxiliary expert heads (dashed path).

Preprint.

1 Introduction

Computational fluid dynamics is central to aerodynamic design. Solving Reynolds-averaged Navier–Stokes (RANS) equations for steady turbulent flows [1] on a discretisation of the domain allows engineers to evaluate a candidate geometry's pressure, velocity and turbulence fields. Evaluating many candidates this way is computationally expensive, which limits how many a design loop can explore [2, 3].

Deep learning has recently emerged as a surrogate [4], where a learned model replaces the solution with a direct map from the geometry, the boundary data and the flow parameters to the solution fields, computed in one forward pass and orders of magnitude faster at inference. Physics-informed networks encode the governing equations in the training objective, so the output satisfies the PDE residual and the boundary conditions [5, 6]. This approach requires an explicit closure model, which is restrictive for industrial RANS turbulence. Neural operators instead learn the input-output map from simulation data to achieve discretisation invariance [7, 8]. The Fourier neural operator [9] approximates the kernel by a pointwise multiplication in the Fourier domain, and extensions to irregular domains use message passing [10]. Mesh deformation [11], or message passing directly on the mesh [12], local kernels approximate the global coupling of elliptic problems poorly, and Fourier bases lose accuracy on complex geometries. Transformer solvers address this coupling directly, at the cost of a quadratic attention complexity [13–16]. Among these, Transolver [17] assigns mesh points to physics slices and attends among the resulting tokens, decoupling cost from mesh size. These architectures share a common trait, in that information is mixed through a single mechanism.

A separate line of work grounds the architecture in the discrete differential operators of the mesh, using discrete exterior calculus [18]. In HodgeNet [19], hodge stars are learned as diagonal matrices and in DiffusionNet as learned diffusion times [20]. HodgeFormer [21] proposes attention-parametrised stars on all three cochain degrees. In all of these constructions, the learned operator lies on the *prediction* path, computing the field directly. Since a discrete Hodge Laplacian $L_0 = \star_0^{-1} d_0^\top \star_1 d_0$ is a Dirichlet-energy operator, a layer built on it smooths the field it predicts.

Sparsely gated mixtures of experts are normally used to scale capacity at roughly constant compute, with a router implemented as a linear probe on the token embedding, under which the specialisation the experts acquire is not fixed by the construction [22, 23]. Geometry-gated mixtures of experts for multi-scale PDE problems are available in the literature. GNOT [15] gates a mixture of experts with a network that takes query coordinates as input, and GAOT [24] combines a ladder of scale-specific branches with a learned per-point softmax gate. We adopt this pattern and use a Hodge-Laplacian controller, and propose a mixture of plain transformer experts, each biased towards one spectral scale. To bind each expert to a scale, we supervise it with a second, diagonal-star Hodge Laplacian applied to the training target, which decomposes the target into a sequence of components

The contributions of this paper can be summarized as follows: (i) We discuss a Gaussian star construction (section 3.5.1) that is symmetric positive definite whenever its neighbour support is mutual, and under which the assembled operator then has a real non-negative spectrum. (ii) We propose an architecture that uses the learned discrete Hodge Laplacian to compute routing weights. (iii) We introduce a supervision scheme in which a second, diagonal-star learnable Laplacian decomposes the target into a sequence of components (section 3.7). (iii) We evaluate the approach on the official AirFRANS split against a Transolver baseline, trained natively on the same working mesh with identical data, loss and metric code (section 4.2). Section S1 lists every symbol used below, with its type and its meaning. The remainder of this paper is organized as follows: section 2 reviews related work, section 3 presents the method, section 4 describes the simulation setup and reports the results, and section 5 discusses the lift gap and the limits of the comparison.

2 Related Work

Physics-informed and data-driven surrogates. Physics-informed neural networks learn PDE solution fields by incorporating the governing equations and boundary conditions into training [5, 6]. Physics-informed neural operators extend this approach to learning solution operators across families of PDE inputs [25]. For RANS flows, this requires specifying the mean-flow equations and a turbulence closure. Our model learns directly from simulated solution fields, following data-driven

airfoil surrogates [4] and the AirFRANS benchmark [1]. Related work extends data-driven CFD modelling to high-fidelity automotive aerodynamics [26].

Neural operators on irregular domains. Neural operators learn maps between function spaces using parameters shared across discretisations [7, 8]. DeepONet represents an operator through branch and trunk networks [27], whereas the Fourier neural operator (FNO) uses learned spectral multipliers to implement global convolutions [9]. Graph kernel networks [10], learned coordinate deformations (Geo-FNO) [11], and graph-to-grid encoding with a latent FNO (GINO) [28] accommodate irregular sampling and variable geometry. MeshGraphNets instead learn local interactions along mesh edges [12]. For local message passing, long-range communication depends on depth or additional multiscale mechanisms where spectral and latent-grid methods provide alternative ways to exchange global information. These choices concern how features propagate, rather than explicitly supervising separate predictors at different mesh spectral scales.

Transformer PDE solvers. Dense attention enables global interactions but has quadratic cost in the number of mesh points. Galerkin attention [13], OFormer [14], GNOT [15], and FactFormer [16] use linear or factorised attention, while inducing-point and anchored-branched architectures process reduced token sets [29, 30]. Transolver [17] softly assigns points to learned physics slices, applies attention among slice tokens, and maps the result back to the points. For a fixed slice count, its cost grows linearly with mesh size. We use Transolver as the principal baseline under a shared remeshing and training protocol. Our dense transformer experts retain quadratic attention cost; the proposed controller addresses expert specialisation rather than removing this scaling limitation.

Geometric gating and multiscale mixtures. Sparsely gated mixtures of experts increase capacity by activating only a subset of experts per token [22, 23]. Geometric and multiscale gating already appear in PDE operator learning: GNOT mixes feed-forward experts using query-coordinate-dependent weights, interpreting the gate as soft domain decomposition [15]. GAOT combines graph operator branches at different neighbourhood radii through learned softmax weights, together with a latent transformer processor [24]. Our contribution builds on this precedent by using a mesh Hodge-Laplacian controller to mix full transformer experts at each vertex, and by supervising auxiliary expert heads with mesh spectral bands. All experts are evaluated, so the mixture is dense. Fourier-feature bandwidths bias the expert inputs towards different spatial scales [31]; band supervision supplies an additional constraint on their auxiliary predictions.

Learned differential operators and spectral analysis. Discrete exterior calculus separates incidence-based topology from metric information encoded in Hodge stars [18]. Hodge Laplacians extend graph differential operators to higher-degree cochains [32, 33].The seminal work of HodgeNet learns discrete Hodge stars for spectral geometry processing [19], DiffusionNet learns diffusion times for surface feature propagation [20], and DiffGCN constructs graph convolutions from differential operators [34]. HodgeFormer parameterises stars through local attention to construct operators on vertices, edges, and faces [21]. Related neural PDE models use structure-preserving finite element formulations [35], Hodge-decomposed solution spaces [36], or cross-dimensional interactions on cell complexes [37]. We use the learned Hodge operator to compute routing weights, while transformer experts produce the predictive features. A separate diagonal-star vertex Laplacian supplies training targets through successive smoothing differences and learned atom-to-expert assignments (section 3.7). This construction relates to graph spectral filtering and wavelets [38, 39], and preserves the sum of the target bands exactly. These bands separate spatial scales of vertex fields; they are not the exact, coexact, and harmonic components of a Hodge decomposition.

3 Methodology

3.1 Overview

We propose a mixture of transformer experts for steady-PDE prediction on unstructured meshes, with per-vertex weights computed by a learnable Laplacian controller. The model predicts velocity components u_x, u_y, pressure p, and turbulent kinematic viscosity ν_t at the vertices of an unstructured mesh. It consists of a shared vertex embedding, eight transformer experts, a Hodge controller, and a field decoder (fig. 1). A separate analyzer constructs auxiliary supervision targets during training.

The prediction path consists of a shared vertex embedding, a Hodge controller, transformer experts, and a field decoder. The controller produces routing weights from vertex, edge, and face features

together with the mesh incidence matrices and neighborhood indices. Each expert combines the shared embedding with fixed Fourier positional features and produces both a latent representation and an auxiliary band prediction. The routing weights combine the latent representations, which the decoder maps to the four output fields. During training, a separate diagonal-star vertex Laplacian smooths the reference fields to construct telescoping atoms. A learned assignment combines these atoms into expert-specific supervision targets. This analyzer is absent from inference, and its conductances receive no gradients from the training loss because the atoms are detached before assignment.

The controller uses two Hodge-former layers [21] operating on vertices, edges, and faces. Their implementation is called the **PSD-Hodge layer**, reflecting the intended positive-semidefinite (PSD) construction. The spectral guarantees depend on symmetry of the effective neighborhoods and compatibility of the star operators, as discussed in sections 3.5.1 and 3.5.2.

3.2 Simplicial complex and neighborhoods

The mesh is the simplicial complex $\mathcal{M} = (V, E, F)$, with $N = |V|$, $N_e = |E|$, and $N_f = |F|$. The signed incidence matrices supplied to the model are $d_0 \in \mathbb{R}^{N_e \times N}$ and $d_1 \in \mathbb{R}^{N_f \times N_e}$, with non zero entries -1 or $+1$. The operators map vertex values to edges and edge values to faces, respectively.

For each mesh element i, its neighborhood is the set of elements of the same type whose features contribute to the learned star at i. Thus, a vertex neighborhood contains vertices, an edge neighborhood contains edges, and a face neighborhood contains faces. We call this set the *operator support* because it specifies the entries that may be nonzero in row i of the star matrix. The stored neighbor list is $\mathcal{N}^{\text{full}}(i)$, while $\mathcal{N}(i) \subseteq \mathcal{N}^{\text{full}}(i)$ denotes the subset used by the controller. Vertex and edge neighbor lists are initially generated from graph-hop rings, with at most eight hops.

The controller uses the first $n_{vcap} = 64$ entries of each stored vertex list and the first $n_{ecap} = 32$ entries of each stored edge list; the reported vertex-only controller uses an edge cap of 64. These lists are ordered by element index after symmetrization, so the retained entries are not necessarily the nearest neighbors. Truncation can also remove the reverse of a retained neighbor relation. Each face neighborhood contains only the face itself. Unused positions in the stored lists are padded with -1; these positions are not neighbors and do not contribute to the operator.

3.3 Input features

The input feature are $x_v \in \mathbb{R}^{N \times c_v}$, $x_e \in \mathbb{R}^{N_e \times c_e}$, and $x_f \in \mathbb{R}^{N_f \times c_f}$. The vertex array contains nine columns:

$$x_v^{\text{raw}} = [\text{pos}_1^{\text{raw}}, \text{pos}_2^{\text{raw}}, A_v^{\text{dual}}, u_{\text{in}}, v_{\text{in}}, \text{SDF}, n_x, n_y, \text{is_wall}], \tag{1}$$

where $\text{pos}_1^{\text{raw}}, \text{pos}_2^{\text{raw}}$ are physical coordinate columns, A_v^{dual} is the barycentric dual area, computed as one third of the sum of incident triangle areas, and SDF denotes signed distance. The columns $u_{\text{in}}, v_{\text{in}}$ contain inlet velocity components, n_x, n_y contain normal components, and is_wall is the wall indicator. Every column, including the wall indicator, is standardized using training-set statistics. Edge and face features are not standardized at dataset level.

The standardized position columns are denoted by $\text{pos} \in \mathbb{R}^{N \times 2}$ and are used to evaluate the expert Fourier features. The prediction and controller embeddings applied to these input arrays are described in section 3.4.

The freestream speed is obtained from the inlet velocity components at the first vertex:

$$U_\infty = \sqrt{u_{\text{in}}^2 + v_{\text{in}}^2}. \tag{2}$$

For velocity components u_x, u_y, pressure p, and turbulent kinematic viscosity ν_t, the dataset applies

$$\widetilde{y} = \left[\frac{u_x}{U_\infty}, \frac{u_y}{U_\infty}, \frac{p}{U_\infty^2}, \frac{\nu_t}{U_\infty}\right], \qquad y_{vc} = \frac{\widetilde{y}_{vc} - \mu_c^{\text{norm}}}{s_c^{\text{norm}} + 10^{-8}}. \tag{3}$$

Here v indexes vertices, c indexes output channels, and $\mu_c^{\text{norm}}, s_c^{\text{norm}}$ are the mean and standard deviation of the scaled training targets for channel c, computed across vertices and reused for all training and test cases. The standardized target is $y \in \mathbb{R}^{N\times 4}$ and the model output is $\hat{u} \in \mathbb{R}^{N\times 4}$. Physical reference fields are denoted by y^{phys}.

The turbulent kinematic viscosity target remains linear, without a logarithmic transformation. Dividing ν_t by U_∞ leaves units of length. A dimensionless target therefore also requires a reference length, which is not explicit in this scaling.

3.4 Input Embedding

The prediction and controller branches use separate input embeddings. The prediction branch maps the input features to one token per vertex through a shared two-layer perceptron with GELU activation:

$$h = W_2\text{GELU}(W_1 x_v) \in \mathbb{R}^{N\times d}, \qquad d = 192. \tag{4}$$

All eight experts receive the same token array h. The controller uses a separate neighbor embedding for vertex, edge, and face features. It first applies normalized adjacency aggregation to the vertex and edge inputs, then maps each stream through its own three-layer perceptron, with ReLU after every linear layer. Face features pass directly through their perceptron. The resulting streams are $h_v \in \mathbb{R}^{N\times d_c}$, $h_e \in \mathbb{R}^{N_e\times d_c}$, and $h_f \in \mathbb{R}^{N_f\times d_c}$.Layer normalization is applied inside the subsequent HodgeFormer blocks.

3.5 Hodge controller

3.5.1 Learned star operators

The controller learns Hodge Stars that combine features within the neighborhood of each mesh element. Using the streams defined in section 3.4, each head first projects the normalized features to $\varphi_i = W_\varphi \text{LN}(h_s)_i$, where $s \in \{v, e, f\}$ and $\varphi_i \in \mathbb{R}^{d_h}$. These projected features determine the interaction between element i and each neighbor $j \in \mathcal{N}(i)$ through a Gaussian kernel:

$$w_{ij} = \exp\left(-\frac{\|\varphi_i - \varphi_j\|_2^2}{2\sigma^2}\right), \qquad \sigma = \text{softplus}(\gamma) + 10^{-8}. \tag{5}$$

Neighbors with similar projected features receive larger weights, while the learned bandwidth σ controls how quickly the weights decrease with feature distance.

The matrix Ω collects these weights on the effective support and is zero elsewhere, including at padded entries. We normalize the weights using the row sums $D_i = \sum_j \Omega_{ij}$, with $D = \text{diag}(D_i)$, and combine the resulting operator with the identity:

$$\hat{S}_{ij} = \frac{\Omega_{ij}}{\sqrt{D_i D_j + 10^{-8}}}, \qquad \star x = (1-\beta)x + \beta\hat{S}x, \qquad \beta = \tfrac{1}{2}\,\text{sigmoid}(\alpha). \tag{6}$$

The identity term retains the element's own values, while $\hat{S}x$ aggregates values from its neighbors. The learned coefficient β controls their relative contribution and remains below $\frac{1}{2}$.

The spectral properties of this construction depend on the neighborhood relation. A neighborhood is reciprocal when $j \in \mathcal{N}(i)$ implies $i \in \mathcal{N}(j)$. Under this condition, the ideal normalized matrix $D^{-1/2}\Omega D^{-1/2}$ is symmetric and has eigenvalues in $[-1, 1]$. The star formed with this ideal normalization is therefore symmetric positive-definite (SPD), with eigenvalues in $[1-2\beta, 1]$. This guarantee requires reciprocity of the effective support used by the operator. The configured caps and truncation can break reciprocity, for an asymmetric support, the identity contribution alone does not establish symmetry or the SPD property.

Face neighborhoods contain only the face itself. For such a support, `psd_star` uses a diagonal alternative that allows the star to depend on the face features:

$$z_i = \frac{\|\varphi_i\|_2^2}{d_h\sigma^2}, \qquad (\star_2 x)_i = (1 - \beta + \beta e^{-z_i})x_i, \tag{7}$$

Here $d_h = d_c/H$ is the kernel-feature dimension, with controller width d_c and head count H. This star scales each face independently, and its diagonal remains positive even if the exponential underflows to zero. The learned scaling is not constrained to equal a physical face area or its reciprocal. Stars labeled as inverse operators have their own learnable parameters and they are not calculated matrix inverses.

3.5.2 Operator composition

We denote the learned operators on vertices, edges, and faces by L_v, L_e, and L_f. For one attention head, let x_v, x_e, x_f denote the normalized latent features supplied to that head. The Gaussian Hodge Stars defined above give

$$\begin{aligned} L_v &:= \star_0^{-1}(x_v) \cdot d_0^T \cdot \star_1(x_e) \cdot d_0, \\ L_e &:= d_0 \cdot \star_0^{-1}(x_v) \cdot d_0^T \cdot \star_1(x_e) + \star_1^{-1}(x_e) \cdot d_1^T \cdot \star_2(x_f) \cdot d_1, \\ L_f &:= d_1 \cdot \star_1^{-1}(x_e) \cdot d_1^T \cdot \star_2(x_f). \end{aligned} \tag{8}$$

The vertex operator reads vertex and edge features, the edge operator reads all three feature arrays, and the face operator reads edge and face features. $\star_0^{-1}$ and $\star_1^{-1}$ have dedicated learnable parameters. The two terms of L_e act on the same projected edge values and their outputs are added. The head outputs are concatenated and projected back to the embedding dimension, as described below.

If the stars in L_v are symmetric positive definite, then $Q = d_0^T \star_1 (x_e) d_0$ is symmetric positive semidefinite and

$$L_v = \star_0^{-1}(x_v) Q \sim \star_0^{-1/2}(x_v) Q \star_0^{-1/2}(x_v). \tag{9}$$

Here $\star_0^{-1/2}(x_v)$ denotes the symmetric square root of the independently learned $\star_0^{-1}(x_v)$. Under this assumption, L_v has a real, nonnegative spectrum, although it is generally not symmetric in the Euclidean inner product. The independently parameterized stars in L_e do not establish the shared-metric compatibility of a conventional Hodge Laplacian, so positive definiteness of the individual stars alone does not guarantee a real, nonnegative spectrum for their summed edge operator. Projections, nonlinearities, and residual updates also mean that the complete neural block is not a PSD linear operator.

3.5.3 Overall Controller Architecture

The controller applies two HodgeFormer[21] layers to the width-64 streams from section 3.4, followed by a vertex readout. Each layer has four heads of width 16 and feedforward width 256. Layer and head indices on the projections and stars are omitted below for readability. Their parameters are independent. After the second HodgeFormer layer, the controller reads only h_v and computes

$$\ell = \mathrm{Linear}_{64\to 8}(\mathrm{GELU}(\mathrm{Linear}_{64\to 64}(\mathrm{LN}(h_v)))) \in \mathbb{R}^{N\times K}, \qquad K = 8. \tag{10}$$

The mixture module converts these logits into routing weights through a softmax without temperature scaling. With the batch axis included, the logit tensor has shape $(B_{\mathrm{batch}}, N, K)$.

3.6 Transformer experts and decoder

Each expert receives an input constructed from the shared vertex embedding h and processes it through transformer blocks with global self-attention over the vertices. Separate output heads produce the representation $\mathrm{proj}_k(z_k)$ and the auxiliary band prediction $\hat{b}_k$. The controller weights combine the expert representations as $h_{\text{mix}}(v) = \sum_{k=1}^{K} g_k(v)\,\mathrm{proj}_k(z_k)(v)$. The decoder forms field tokens from the mixed representation using learned field embeddings, applies self-attention across the fields at each vertex, and maps the resulting tokens to the predictions $\hat{u}$ through a shared readout. The auxiliary predictions are supervised against the assigned band targets and do not enter the final field reconstruction. The expert architecture, mixture, and decoder are specified in sections S3.5 and S3.6.

3.7 Auxiliary band targets

3.7.1 Analyzer

The analyzer constructs conductances from unstandardized edge features:

$$w_e = \mathrm{softplus}(\mathrm{MLP}_e(x_e)) + \epsilon_w, \qquad \epsilon_w = 10^{-4}, \tag{11}$$

$$\star_1 = \mathrm{diag}(w), \qquad \mu = |d_0|^\top w, \qquad \star_0 = \mathrm{diag}(\mu), \qquad L_0 = \star_0^{-1} d_0^\top \star_1 d_0. \tag{12}$$

Here w collects the conductances w_e, and μ collects the degrees $\mu_v = \sum_{e \ni v} w_e$. These diagonal stars are separate from the controller stars in section 3.5. The analyzer's $\star_0^{-1}$ is the reciprocal diagonal mass matrix.

The perceptron MLP_e has widths $c_e \to 64 \to 1$ with GELU activation. Vertex masses are clamped below by ϵ_w. With unit incidence and `learn_mass=false`, μ_v is the weighted graph degree. The resulting random-walk L_0 is self-adjoint and PSD in the degree-weighted inner product. It is generally not symmetric in the Euclidean inner product, but is similar to a symmetric PSD normalized Laplacian and has eigenvalues in $[0, 2]$.

One smoothing step is $\bar{y} \leftarrow (I - \tau L_0)\bar{y}$, with $\tau = \frac{1}{2}$. The spectrum of $I - \tau L_0$ lies in $[0, 1]$, so repeated application suppresses higher graph frequencies. Power iteration is available for diagnostics but is not used in band construction.

3.7.2 Decomposition and assignment

The ladder has $L = 8$ rungs,

$$(m_0, \ldots, m_{L-1}) = (2, 4, 6, 8, 12, 16, 24, 32), \qquad \bar{y}^{(j)} = (I - \tau L_0)^{m_j} y, \quad j = 0, \ldots, L-1. \tag{13}$$

The $M = L + 1 = 9$ atoms are

$$a_0 = \bar{y}^{(L-1)}, \qquad a_i = \bar{y}^{(L-1-i)} - \bar{y}^{(L-i)} \quad (1 \le i \le L-1), \qquad a_{M-1} = y - \bar{y}^{(0)}. \tag{14}$$

They sum to y algebraically, subject to floating-point rounding. These are overlapping spectral-filter components rather than orthogonal projections onto disjoint frequency intervals.

The assignment logits are $\theta \in \mathbb{R}^{K \times M}$, and

$$P_{km} = \frac{\exp(\theta_{km}/T)}{\sum_{k'=1}^{K} \exp(\theta_{k'm}/T)}, \qquad T = 1, \qquad t_k = \sum_{m=0}^{M-1} P_{km} a_m. \tag{15}$$

Each column of P sums to one, so $\sum_{k=1}^{K} t_k = y$ algebraically. Each expert's auxiliary prediction $\hat{b}_k$ is supervised against the four-channel target t_k. The analyzer atoms are detached before forming these targets, so auxiliary supervision updates the assignment logits without updating the analyzer conductances. The learned assignment is not constrained to preserve a strict low-to-high expert ordering.

Using expert indices $k = 1, \ldots, K$, the constructor initially favors expert k for atom a_{k-1}, with no favored expert for a_{M-1}. The later global Xavier initialization overwrites these logits.

3.8 Training Objective

The training objective combines field reconstruction, auxiliary band prediction, routing balance, and a floor on assigned-band energy:

$$\begin{aligned} \mathcal{L} = \mathcal{L}_{\mathrm{vol}} + \mathcal{L}_{\mathrm{surf}} + \lambda_b \,\mathrm{mean}_{v,c,k} (\hat{b}_{vck} - t_{vck})^2 \\ + \lambda_{\mathrm{bal}} K \sum_{k=1}^{K} \left(\bar{g}_k - \frac{1}{K} \right)^2 + \lambda_{\mathrm{floor}} \sum_{k=1}^{K} [\mathrm{ReLU}(\rho - p_k)]^2. \end{aligned} \tag{16}$$

Here $\bar{g}_k$ is the routing weight averaged over batch and vertices, and p_k is the normalized squared norm of expert k's assigned target. We use $(\lambda_b, \lambda_{\mathrm{bal}}, \lambda_{\mathrm{floor}}) = (0.5, 0.01, 1.0)$ and $\rho = 0.02$. The field terms are mean squared errors on disjoint non-wall and wall vertex sets, using residuals

$$e_{vc} = \frac{\hat{u}_{vc} - y_{vc}}{\max(\mathrm{std}_v(y_{vc}), 0.1)}, \qquad \mathcal{L}_{\mathrm{vol}} = \mathrm{mean}_{v \in V_{\mathrm{vol}}, c} \, e_{vc}^2, \quad \mathcal{L}_{\mathrm{surf}} = \mathrm{mean}_{v \in V_{\mathrm{surf}}, c} \, e_{vc}^2. \tag{17}$$

The band term supervises each auxiliary expert prediction against the target of eq. (15). For $E_k = \sum_{v,c} t_{vck}^2$, the energy share is $p_k = E_k / \max(\sum_{k'} E_{k'}, 10^{-9})$. These shares describe overlapping

Table 1: Comparison on the official AirFRANS $800/200$ split. Every row is evaluated on the same ≈ 4.9k-vertex working meshes and evaluation metrics. Baselines use their full input-feature arms; training and checkpoint-selection differences are specified in section 4.1. *Mesh use* names how mesh structure reaches the prediction: *none* for a point-set model, *operator* when a learned Hodge Laplacian computes the fields, *router* when it computes only the mixing weights. C_L is the relative error of the mesh-native lift over the 200 test cases and ρ_L its Spearman correlation with the reference.

model	params	mesh use	rel-$L_2 \downarrow$	$R^2 \uparrow$	Volume$\downarrow$	Surf$\downarrow$	$C_L \downarrow$	$\rho_L \uparrow$
Transolver [17]	6.00M	none	0.0436	0.9953	0.00265	**0.00143**	**0.0103**	**0.99989**
PointNet [43]	0.08M	none	0.0689	0.9904	0.00451	0.00440	0.0149	0.99965
Pointwise MLP	0.02M	none	0.1743	0.9404	0.03576	0.03969	0.0860	0.99338
PSD-star HodgeFormer [21]	0.47M	operator	0.0800	0.9885	0.01028	0.01048	0.0681	0.99883
Hodge-routed MoE (ours)	3.02M	router	**0.0300**	**0.9983**	**0.00140**	0.00164	0.0225	0.99988

targets rather than an orthogonal partition of field energy. Analyzer atoms are detached before assignment, so the band loss and floor penalty train the assignment without updating the analyzer conductances. Band-energy entropy is logged but is not included in the objective.

4 Experiments

4.1 Simulation Setup

AirFRANS [1] provides node positions, input features, and four solution fields. We generate anisotropic triangular working meshes with gmsh [40], using approximately 4.9k vertices with refinement near the airfoil, and interpolate the reference fields onto them. Coordinates are embedded at $z = 0$ to construct the incidence matrices and geometric features.

The nine vertex inputs are defined in section 3.3. They encode geometry, inlet velocity, signed distance, supplied normals, and wall membership. Boundary conditions enter through these features; wall vertices define the surface loss and Surf metric. Edge features also determine the analyzer conductances. The reported vertex-only controller uses edge features in its vertex operator, while the VEF variant additionally updates edge and face streams.

We use the official $800/200$ split. Each training access selects one of up to eight available remeshes; evaluation uses a fixed mesh per case. Targets are scaled by the sample's freestream speed and standardized using training statistics, as specified in eq. (3).

Per-field relative L_2 and R^2 are evaluated in physical units. Volume and Surf are globally normalized mean squared errors on non-wall and wall vertices, respectively. We also report mesh-native relative lift-coefficient error and its Spearman correlation across test cases. All models are evaluated against interpolated targets on the same working meshes; these results are not directly comparable with original-mesh benchmark scores.

Our checkpoint is selected from 20 checkpoints using test-set error, whereas the baselines use validation-based selection. Our models train on all 800 training cases; the baselines reserve 80 for validation. These protocol differences limit attribution of the reported gains to architecture alone.

Training uses AdamW [41] and a one-cycle schedule [42] for 400 epochs, with one mesh per batch. We use `bfloat16` autocast and `fp32` incidence and analyzer operations on one NVIDIA RTX 4090 with 24 GB memory. Full settings are given in section S4.

4.2 Results

Table 1 compares the models on the official split. Our model has the lowest relative L_2 error and the lowest interior mean squared error of the five, 31% and 47% below Transolver at half its parameter count. Transolver has the lower error on the two wall quantities, by a factor of 1.14 on the surface error and 2.2 on the lift coefficient. The fourth row is a HodgeFormer surrogate, which keeps the learned Hodge Laplacian in the prediction path. The per-field breakdown is given in full in the supplementary material: our model has the lowest relative L_2 error on three of the four fields and the second lowest on u_y, and the largest difference is on the eddy viscosity, the least accurate output of every model, where our error is 46% below Transolver's. The aggregate is the mean over the four fields. The fixed reference decomposition is illustrated for pressure in fig. S5. The design was

selected through ablations over the source of the gate, the width of the predictor, fixed against learned band boundaries, the guard against an empty band, and the number of cochain degrees the controller operates on, given in the supplementary material.

Figure 2 places the four models beside the CFD reference on a common colour scale. The models separate more on the turbulent viscosity than on the pressure: the mean relative L_2 over the 200 test cases is 0.063, 0.116, 0.144 and 0.155 on ν_t for (a)–(d), against 0.026, 0.030, 0.066 and 0.085 on p. The reference carries fine-scale structure in the ν_t wake, and the ordering of the panels on that field follows the ordering of those errors.

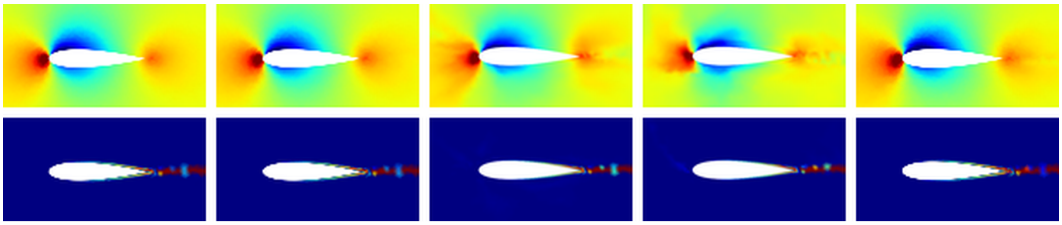


Figure 2: Pressure p (top) and turbulent viscosity ν_t (bottom) on the reference colour scales; left to right, the CFD reference, (a) our model, (b) Transolver, (c) PointNet, and (d) the best PSD-star HodgeFormer. Values beneath each model panel are mean relative L_2 errors over the 200 test cases. The models differ more on ν_t than on p; the reference panel shows fine-scale structure in the ν_t wake.

Figure 3 shows one test case at the reported checkpoint, with the reference, prediction, and absolute error for each field. Its per-field relative L_2 errors are 0.0047, 0.0260, 0.0281, and 0.0676, compared with test-set means of 0.0056, 0.0259, 0.0255, and 0.0629. This single case does not establish representativeness of the test set. Visible errors occur near the airfoil and along the turbulent-viscosity wake.

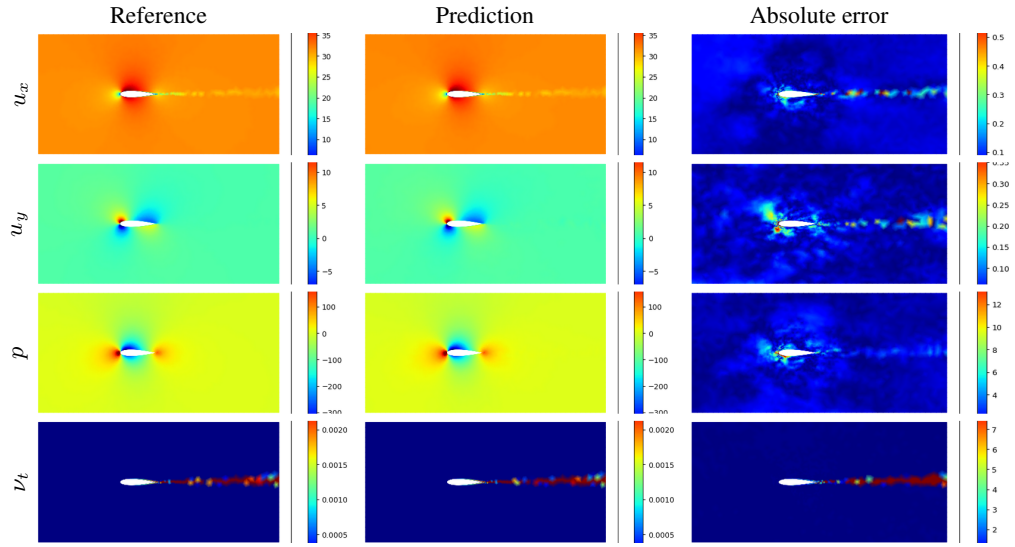


Figure 3: One test case at the reported checkpoint (epoch 380). Columns are the reference, the prediction and the absolute error; rows are u_x, u_y, p and ν_t. All panels are on the model's own working mesh in physical units, with the reference and the prediction sharing each field's colour scale. The per-field relative L_2 for this case is 0.0047, 0.0260, 0.0281 and 0.0676. Test-set means are listed in table S7; the panel illustrates one case rather than a test-set average.

5 Discussion and Conclusion

We propose a Hodge-routed mixture of transformer experts for steady-PDE prediction on unstructured meshes. A learned Hodge Laplacian computes the routing weights, and a separate analyzer constructs multiscale supervision targets. On the AirFRANS working mesh, mean relative L_2 and interior mean

squared errors decrease by 31% and 47% relative to Transolver, from 0.0436 to 0.0300 and from 0.00265 to 0.00140, respectively. The largest per-field improvement is in turbulent viscosity, whose relative L_2 error decreases by approximately 46%, from 0.1158 to 0.0629 (table S7). Introducing the Hodge controller into the learned-band mixture reduces mean relative L_2 error from 0.0345 to 0.0300 (table S6), with larger additional gains in velocity and pressure than in turbulent viscosity (table S7).

The mean relative L_2 error for pressure is lower than Transolver's, while surface and relative lift-coefficient errors are higher: 0.00164 versus 0.00143 and 0.0225 versus 0.0103, respectively (table 1). Both models retain lift rank correlations above 0.9998, preserving case ordering more accurately than force magnitudes. Pressure errors along the airfoil contour therefore require further examination.

We follow Transolver's evaluation format on the official AirFRANS $800/200$ split, with one training run per configuration and evaluation on a remeshed working mesh of approximately 4.9K vertices. The results do not establish accuracy at the original CFD resolution or robustness across random seeds. The analyzer's algebraic target reconstruction holds independently of the controller's conditional spectral guarantee. The implementation does not enforce the reciprocal neighborhoods required by that guarantee, and prediction accuracy does not validate it. Every expert is evaluated, and dense attention scales quadratically with mesh size.

References

[1] Florent Bonnet et al. AirFRANS: High fidelity computational fluid dynamics dataset for approximating Reynolds-averaged Navier–Stokes solutions. In *Adv. Neural Inform. Process. Syst.*, 2022.

[2] Steven L. Brunton, Bernd R. Noack, and Petros Koumoutsakos. Machine learning for fluid mechanics. *Annual Review of Fluid Mechanics*, 52:477–508, 2020.

[3] Ricardo Vinuesa and Steven L. Brunton. Enhancing computational fluid dynamics with machine learning. *Nature Computational Science*, 2(6):358–366, 2022.

[4] Nils Thuerey, Konstantin Weißenow, Lukas Prantl, and Xiangyu Hu. Deep learning methods for Reynolds-averaged Navier–Stokes simulations of airfoil flows. *AIAA Journal*, 58(1):25–36, 2020.

[5] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

[6] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.

[7] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *J. Mach. Learn. Res.*, 24(89):1–97, 2023. doi: 10.5555/3648699.3648788.

[8] Kamyar Azizzadenesheli, Nikola Kovachki, Zongyi Li, Miguel Liu-Schiaffini, Jean Kossaifi, and Anima Anandkumar. Neural operators for accelerating scientific simulations and design. *arXiv preprint arXiv:2309.15325*, 2023.

[9] Zongyi Li et al. Fourier neural operator for parametric partial differential equations. In *Int. Conf. Learn. Represent.*, 2021.

[10] Zongyi Li et al. Neural operator: Graph kernel network for partial differential equations. In *ICLR Workshop on Integration of Deep Neural Models and Differential Equations*, 2020.

[11] Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for PDEs on general geometries. *J. Mach. Learn. Res.*, 24(388):1–26, 2023.

[12] Tobias Pfaff and others. Learning mesh-based simulation with graph networks. In *Int. Conf. Learn. Represent.*, 2021.

[13] Shuhao Cao. Choose a transformer: Fourier or Galerkin. In *Adv. Neural Inform. Process. Syst.*, 2021.

[14] Zijie Li, Kazem Meidani, and Amir Barati Farimani. Transformer for partial differential equations' operator learning. *Trans. Mach. Learn. Res.*, 2022. arXiv:2205.13671.

[15] Zhongkai Hao et al. GNOT: A general neural operator transformer for operator learning. In *Int. Conf. Mach. Learn.*, 2023.

[16] Zijie Li, Dule Shu, and Amir Barati Farimani. Scalable transformer for PDE surrogate modeling. *arXiv preprint arXiv:2305.17560*, 2023.

[17] Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for PDEs on general geometries. In *Int. Conf. Mach. Learn.*, 2024.

[18] Mathieu Desbrun, Anil N. Hirani, Melvin Leok, and Jerrold E. Marsden. Discrete exterior calculus. *arXiv preprint math/0508341*, 2005.

[19] Dmitriy Smirnov and Justin Solomon. HodgeNet: Learning spectral geometry on triangle meshes. *ACM Trans. Graph.*, 40(4):1–11, 2021.

[20] Nicholas Sharp, Souhaib Attaiki, Keenan Crane, and Maks Ovsjanikov. DiffusionNet: Discretization agnostic learning on surfaces. *ACM Trans. Graph.*, 41(3):1–16, 2022.

[21] Akis Nousias and Stavros Nousias. HodgeFormer: Transformers for learnable operators on triangular meshes through data-driven hodge matrices. In *IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV)*, 2026. arXiv:2509.01839.

[22] Noam Shazeer et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Int. Conf. Learn. Represent.*, 2017.

[23] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[24] Shizheng Wen, Arsh Kumbhat, Levi Lingsch, Sepehr Mousavi, Yizhou Zhao, Praveen Chandrashekar, and Siddhartha Mishra. Geometry aware operator transformer as an efficient and accurate neural surrogate for PDEs on arbitrary domains. *arXiv preprint arXiv:2505.18781*, 2025.

[25] Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM/IMS J. Data Sci.*, 2024. doi: 10.1145/3648506.

[26] Maurits Bleeker, Matthias Dorfer, Tobias Kronlachner, Reinhard Sonnleitner, Benedikt Alkin, and Johannes Brandstetter. NeuralCFD: Deep learning on high-fidelity automotive aerodynamics simulations. *arXiv preprint arXiv:2502.09692*, 2025.

[27] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021. doi: 10.1038/s42256-021-00302-5.

[28] Zongyi Li, Nikola Kovachki, Chris Choy, Boyi Li, Jean Kossaifi, Shourya Otta, Mohammad Amin Nabian, Maximilian Stadler, Christian Hundt, Kamyar Azizzadenesheli, and Anima Anandkumar. Geometry-informed neural operator for large-scale 3D PDEs. In *Adv. Neural Inform. Process. Syst.*, 2023.

[29] Seungjun Lee and Taegon Oh. Inducing point operator transformer: A flexible and scalable architecture for solving PDEs. In *AAAI*, 2024. doi: 10.1609/aaai.v38i1.27766.

[30] Maurits Bleeker, Matthias Dorfer, Tobias Kronlachner, Reinhard Sonnleitner, Benedikt Alkin, and Johannes Brandstetter. AB-UPT: Scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers. *Trans. Mach. Learn. Res.*, 2025.

[31] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Adv. Neural Inform. Process. Syst.*, 2020.

[32] Lek-Heng Lim. Hodge Laplacians on graphs. *SIAM Review*, 62(3):685–715, 2020.

[33] Michael T. Schaub, Austin R. Benson, Paul Horn, Gábor Lippner, and Ali Jadbabaie. Random walks on simplicial complexes and the normalized hodge 1-Laplacian. *SIAM Review*, 62(2):353–391, 2020. doi: 10.1137/18m1201019.

[34] Moshe Eliasof and Eran Treister. DiffGCN: Graph convolutional networks via differential operators and algebraic multigrid pooling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[35] Benjamin Shaffer, Shawn Koohy, Brooks Kinch, M. Ani Hsieh, and Nathaniel Trask. Structure-preserving learning improves geometry generalization in neural PDEs. *arXiv preprint arXiv:2602.02788*, 2026.

[36] Dongzhe Zheng, Tao Zhong, and Christine Allen-Blanchette. Topology-preserving neural operator learning via hodge decomposition. *arXiv:2605.13834*, 2026.

[37] Lennart Bastian, Samuel Leventhal, Mustafa Hajij, and Tolga Birdal. Topological neural operators. *arXiv preprint arXiv:2606.09806*, 2026.

[38] David I. Shuman, Sunil K. Narang, Pascal Frossard, Antonio Ortega, and Pierre Vandergheynst. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. *IEEE Signal Processing Magazine*, 30(3):83–98, 2013.

[39] David K. Hammond, Pierre Vandergheynst, and Rémi Gribonval. Wavelets on graphs via spectral graph theory. *Applied and Computational Harmonic Analysis*, 30(2):129–150, 2011.

[40] Christophe Geuzaine and Jean-François Remacle. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *Int. J. Numer. Methods Eng.*, 79(11):1309–1331, 2009.

[41] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Int. Conf. Learn. Represent.*, 2019.

[42] Leslie N. Smith and Nicholay Topin. Super-convergence: Very fast training of neural networks using large learning rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, 2019.

[43] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2017.

Supplementary Material

The proposed Hodge-routed mixture of experts uses a controller with vertex, edge, and face updates (VEF). The vertex-only configuration (V) is an additional comparison. The supplementary material gives notation, HodgeFormer background, implementation details, training and evaluation settings, model configurations, and results. The controller comparison reports V and VEF separately. The training curves show the vertex-only run. The analyzer and routing measurements use its checkpoint at epoch 380.

S1 Notation

Table S1 defines the main symbols used in the paper and supplementary material. Shared constants and VEF-specific quantities are listed together. Array dimensions do not include the batch dimension. The symbols x_v, x_e, x_f denote input features in section 3.3 and normalized features used to compute the mesh operators. The dimensions are given with each equation. Controller stars and analyzer stars are separate operators. Superscripts on decoder tokens indicate block depth. Subscripts on assigned targets identify experts. In the embedding notation, $\delta_s = \mathrm{diag}(d_s^\top d_s)$ with $d_v = d_0$ and $d_e = d_1$, and A_s is the normalized adjacency matrix with its diagonal removed. In the HodgeFormer background (section S2.1), h counts attention heads and σ denotes softmax. Elsewhere, these symbols denote the shared embedding and Gaussian bandwidth.

Table S1: Notation. Main symbols grouped by their role. Array dimensions do not include the batch dimension. Uses specific to the HodgeFormer and transformer equations are identified.

symbol	type	meaning	symbol	type	meaning
The discretisation					
$\mathcal{M}$	(V, E, F)	the working mesh as a simplicial complex	$e = (i, j)$	index	an edge with endpoints i, j
V, E, F	sets	vertices, edges, faces	i, j, k, m, s	indices	elements i, j; expert k; atom m; stream s
N, N_e, N_f	integers	$\lvert V\rvert, \lvert E\rvert, \lvert F\rvert$; $N \approx 4900$	c	index	output channel in y_{vc}
v	index	a vertex			
Input features					
x_v	$\mathbb{R}^{N \times c_v}$	standardized vertex inputs	(c_v, c_e, c_f)	$(9, 21, 13)$	channel counts
x_e	$\mathbb{R}^{N_e \times c_e}$	edge geometry inputs, not standardized	pos	$\mathbb{R}^{N \times 2}$	standardized position columns of x_v
x_f	$\mathbb{R}^{N_f \times c_f}$	face geometry inputs, not standardized			
Target and prediction					
u_x, u_y	fields	velocity components	U_∞	scalar	freestream speed of a sample
p	field	kinematic pressure	y	$\mathbb{R}^{N \times 4}$	training target, standardised
ν_t	field	turbulent kinematic viscosity	$\hat{u}$	$\mathbb{R}^{N \times 4}$	standardized field prediction
Discrete operators					
d_0	incidence	$\mathbb{R}^{N_e \times N}$, $(d_0 x)_e = x_j - x_i$	L_0	$\mathbb{R}^{N \times N}$	vertex operator; stars depend on context
d_1	incidence	$\mathbb{R}^{N_f \times N_e}$, $d_1 d_0 = 0$	Q	$\mathbb{R}^{N \times N}$	controller matrix $d_0^\top \star_1 d_0$
$\star_0, \star_1, \star_2$	matrices	stars on 0-, 1-, 2-cochains	M	integer	atom count, $M = 9$
$\star_0^{-1}, \star_1^{-1}$	matrices	controller: independent learned operators			

Table S1, continued

symbol	type	meaning	symbol	type	meaning
Neighbourhoods and supports					
$\mathcal{N}^{\text{full}}(i)$	set	stored neighbors: vertex/edge cap 256, face cap 1	support	pairs	the (i, j) with $j \in \mathcal{N}(i)$
$\mathcal{N}(i)$	set	effective neighbors; caps in table S3	mutual	property	$j \in \mathcal{N}(i) \Rightarrow i \in \mathcal{N}(j)$
n_{vcap}, n_{ecap}	$64, 32$	VEF vertex and edge caps	V, VEF	configurations	vertex-only; vertex, edge, and face updates
The controller (section 3.5)					
ℓ	$\mathbb{R}^{N\times K}$	vertex routing logits	$V^{(i)}$	array	projected values of head i; shape depends on stream
h_v, h_e, h_f	arrays	controller streams, each of width d_c	$W_V^{(i)}, W_{\text{out}}$	matrices	per-head value; per-layer output
d_c, L_c, H	$64, 2, 4$	width, layers, heads	W_a, W_b	matrices	read-out, $d_c \to 64 \to K$
d_h	16	d_c/H, per-head width	$\text{Emb}_s, \text{FFN}_s$	networks	embedding; feed-forward
A_s	matrix	normalized off-diagonal embedding adjacency	$\delta_s, \tilde{D}_s$	vector/matrix	incidence degrees; $\tilde{D}_s = \text{diag}(\delta_s) + I$
The Gaussian star (section 3.5.1)					
φ_i	$\mathbb{R}^{d_h}$	kernel feature, $W_\varphi \text{LN}(h)_i$	N_c	integer	element count in the feature array
W_φ	$\mathbb{R}^{d_h\times d_c}$	shared kernel projection for elements i, j	σ	> 0	bandwidth, $\text{softplus}(\gamma) + 10^{-8}$
w_{ij}	$(0, 1]$	Gaussian kernel weight	β	$(0, \frac{1}{2})$	$\frac{1}{2}\,\text{sigmoid}(\alpha)$
Ω	$\mathbb{R}^{N_c\times N_c}$	w_{ij} on the support, else 0	γ, α	scalars	raw parameters; init $0.5413, -0.85$
D, D_i	matrix/scalar	$D = \text{diag}(D_i)$; $D_i = \sum_j \Omega_{ij}$	A	matrix	unweighted vertex adjacency in fixed-band decomposition
$\hat{S}$	matrix	implemented normalization in eq. (6)			
Experts, gate and decoder (section 3.6)					
K	8	number of experts	W_{in}^k, W_b^k	matrices	expert input; band head
d, d_x	$192, 88$	token width; expert width	proj_k	projection	maps z_k to an $N \times d$ array, with bias
L_x, h_x	$2, 8$	blocks; heads per block	$\hat{b}_k$	$\mathbb{R}^{N\times 4}$	band-head output
h	$\mathbb{R}^{N\times d}$	token embedding, shared	$g(v)$	Δ^K	gate weights at vertex v
z_k, z_k^0	$\mathbb{R}^{N\times d_x}$	output, input of expert k	Δ^K	simplex	K nonnegative weights summing to one
n_F	8	frequencies; sine/cosine give $2n_F$ channels	h_{mix}	$\mathbb{R}^{N\times d}$	combined expert features
σ_k	constants	fixed Fourier scales, 1.0 to 38.0 (table S3)	E_{field}	$\mathbb{R}^{4\times d}$	decoder field embedding
G_k, B_k	$\mathbb{R}^{2\times n_F}$	fixed normal samples; $B_k = \sigma_k G_k$			
The analyzer (section 3.7)					
w_e	$\geq \epsilon_w$	edge conductance	$\bar{y}^{(j)}$	$\mathbb{R}^{N\times 4}$	target after m_j steps

Table S1, continued

symbol	type	meaning	symbol	type	meaning
ϵ_w	10^{-4}	keeps w_e positive	m_j, L	integers	m_j smoothing steps; $L = 8$ levels
μ_v	> 0	incident conductance sum; mass uses floor ϵ_w	a_i	$\mathbb{R}^{N\times 4}$	atom i; $\sum_i a_i = y$
MLP_e	$21 \to 64 \to 1$	edge-conductance score	P, θ	$\mathbb{R}^{K\times M}$	column-normalized assignment and its logits
τ	$\frac{1}{2}$	forward-Euler step	t_k	$\mathbb{R}^{N\times 4}$	band target; $\sum_k t_k = y$
The objective (section 3.8)					
$\mathcal{L}$	scalar	the training objective	$\bar{g}_k$	scalar	mean gate mass of expert k
$\mathcal{L}_{\text{vol}}, \mathcal{L}_{\text{surf}}$	scalars	interior, wall field terms	p_k	$[0, 1]$	assigned-target squared-norm share, denominator floored
Metrics					
rel-L_2	≥ 0	mean case/field relative L_2, physical units	C_L	≥ 0	relative L_2 of lift-coefficient vectors
R^2	≤ 1	coefficient of determination	ρ_L	$[-1, 1]$	Spearman correlation of lift
Volume, Surf	≥ 0	globally normalized MSE, non-wall and wall			
Generic operators					
LN	operator	layer normalisation	$\lambda(\cdot)$	set	the spectrum of a matrix
Drop	operator	dropout, probability 0.1	$\sim$	relation	similarity of matrices
GELU, MHSA	operators	activation; self-attention	$[\,\cdot\,]$	operator	concatenation; axis specified locally
$\mathrm{diag}(\cdot)$	matrix	diagonal from a vector	$\odot$	operator	elementwise product
Additional local notation					
L_v, L_e, L_f	operators	vertex, edge, face operators in eq. (8)	B_{batch}	integer	batch size
x_v^{raw}	$\mathbb{R}^{N\times 9}$	vertex features before standardization	A_v^{dual}	area	barycentric dual vertex area
$\mu_c^{\text{norm}}, s_c^{\text{norm}}$	scalars	per-channel statistics of scaled training targets	$\widetilde{y}$	$\mathbb{R}^{N\times 4}$	freestream-scaled targets
T	1	assignment temperature in eq. (15)	E_k	scalar	$\sum_{v,c} t_{vck}^2$
$\lambda_b, \lambda_{\text{bal}}, \lambda_{\text{floor}}$	scalars	loss weights 0.5, 0.01, 1.0	ρ	0.02	minimum assigned-target norm share
e_{vc}	scalar	normalized field residual in eq. (17)	$V_{\text{vol}}, V_{\text{surf}}$	sets	non-wall and wall vertices
t_{vc}^r, T_v^r	vector/array	decoder field token and four-token array at depth r	$m(v)$	$\mathbb{R}^d$	combined features after normalization and dropout
X, n, D, J, q	local notation	transformer tokens, count, width, heads, head width $q = D/J$	Q_a, K_a, V_a	arrays	query, key, value of transformer head a
A_a, O_a	matrices	head attention weights and output in eq. (S15)	z_i	scalar	diagonal-face kernel score in eq. (7)

S2 HodgeFormer Background

S2.1 HodgeFormer

HodgeFormer [21] extends the Transformer layer to operate on vertex, edge, and face features. Following the notation of the original paper, the feature arrays are

$$x_k \in \mathbb{R}^{n_k \times d_k}, \qquad k \in \{v, e, f\}. \tag{S1}$$

For a common embedding dimension $d_k = d$ and h attention heads, each head has dimension $d_h = d/h$. Learnable projection matrices define the query, key, and value representations,

$$\begin{aligned} Q_k &= x_k W_{Q_k}, \\ K_k &= x_k W_{K_k}, \qquad W_{Q_k}, W_{K_k}, W_{V_k} \in \mathbb{R}^{d \times d_h}. \\ V_k &= x_k W_{V_k}, \end{aligned} \tag{S2}$$

The original HodgeFormer parameterizes a Hodge Star through row-wise softmax attention,

$$\star_k(x_k) = \sigma\left(\frac{Q_k K_k^T}{\sqrt{d_h}}\right), \tag{S3}$$

where $\sigma(\cdot)$ denotes the row-wise softmax. Separate linear maps are used for the inverse Hodge stars. The learned operators are the following products of star and signed incidence matrices:

$$\begin{aligned} L_v &:= \star_0^{-1}(x_v) \cdot d_0^T \cdot \star_1(x_e) \cdot d_0, \\ L_e &:= d_0 \cdot \star_0^{-1}(x_v) \cdot d_0^T \cdot \star_1(x_e) \\ &\quad + \star_1^{-1}(x_e) \cdot d_1^T \cdot \star_2(x_f) \cdot d_1, \\ L_f &:= d_1 \cdot \star_1^{-1}(x_e) \cdot d_1^T \cdot \star_2(x_f). \end{aligned} \tag{S4}$$

For one head, the attention updates apply each operator to the corresponding projected values,

$$\begin{aligned} A_v(x_v, x_e) &= L_v(x_v, x_e) \cdot V_v(x_v), \\ A_e(x_v, x_e, x_f) &= L_e(x_v, x_e, x_f) \cdot V_e(x_e), \\ A_f(x_e, x_f) &= L_f(x_e, x_f) \cdot V_f(x_f). \end{aligned} \tag{S5}$$

The head outputs are joined along the feature dimension. A linear projection restores the embedding width. These operations define multi-head Hodge attention A_{H_l}. The HodgeFormer layer is

$$\begin{aligned} H_l(x_v, x_e, x_f) := G_l\big(&A_{H_l}(x_v, x_e, x_f) \\ &+ (x_v, x_e, x_f)\big), \end{aligned} \tag{S6}$$

where G_l applies separate two-layer feedforward networks to the vertex, edge, and face features. Each network uses a residual connection, which adds its input to its output. These equations describe the original HodgeFormer. The Gaussian-star controller used in this work is specified in section 3.5.

S3 Methodology

This section gives additional details of the controller, expert inputs, transformer blocks, mixture, and field decoder described in section 3. Widths and counts are listed in table S3.

S3.1 Overall Controller Architecture

The proposed VEF controller updates vertex, edge, and face features through two Gaussian-Hodge layers, followed by a vertex readout. The vertex-only configuration is described in section S8. The controller applies two HodgeFormer [21] layers to the vertex, edge, and face feature arrays from section 3.4. Each array has 64 channels. Each layer has four heads of width 16 and feedforward width 256. The projections and stars have separate parameters for each layer and head. These indices are omitted from the equations.

All three feature arrays are normalized before attention. With $x_v = \mathrm{LN}(h_v)$, $x_e = \mathrm{LN}(h_e)$, and $x_f = \mathrm{LN}(h_f)$, one head computes

$$\begin{aligned} A_v(x_v, x_e) &= L_v(x_v, x_e) \cdot V_v(x_v), \\ A_e(x_v, x_e, x_f) &= L_e(x_v, x_e, x_f) \cdot V_e(x_e), \\ A_f(x_e, x_f) &= L_f(x_e, x_f) \cdot V_f(x_f), \end{aligned} \tag{S7}$$

where $V_s(x_s) = x_s W_{V_s}$ is the learned value projection for stream $s \in \{v, e, f\}$. The operators are those of eq. (8).

The four head outputs are joined along the feature dimension. Dropout and a linear projection then return 64 channels. The combined output for feature array s is $A_{H_l,s}$. The layer updates are

$$\begin{aligned} \widetilde{h}_s &= h_s + \mathrm{Drop}(A_{H_l,s}(x_v, x_e, x_f)), \\ h_s^{\mathrm{out}} &= \widetilde{h}_s + \mathrm{Drop}(\mathrm{FFN}_s(\mathrm{LN}(\widetilde{h}_s))), \\ \mathrm{FFN}_s(x) &= \mathrm{Linear}_{256\to 64}(\mathrm{ReLU}(\mathrm{Linear}_{64\to 256}(x))). \end{aligned} \tag{S8}$$

Each feature array has its own normalization layers, projections, and feedforward network. The feedforward updates with residual connections form G_l. Dropout has probability 0.1. The first sublayer applies the mesh operators. The second applies a feedforward network separately to each mesh element. All three attention updates use the features from the start of the layer.

The vertex operator uses edge features through $\star_1$. The edge operator uses vertex and face features through $\star_0^{-1}$ and $\star_2$. The first layer therefore uses face features to update edge features. The second layer can then use those edge features to update vertices. The vertex readout does not directly use the final face update.

After the second HodgeFormer layer, the controller reads only h_v and computes

$$\ell = \mathrm{Linear}_{64\to 8}(\mathrm{GELU}(\mathrm{Linear}_{64\to 64}(\mathrm{LN}(h_v)))) \in \mathbb{R}^{N\times K}, \qquad K = 8. \tag{S9}$$

The controller outputs scores before softmax, called logits. The mixture module converts these scores into routing weights using softmax, without temperature scaling. Including the batch dimension, the score array has shape $(B_{\mathrm{batch}}, N, K)$.

S3.2 Conditional spectral properties

The Gaussian star in section 3.5.1 is symmetric positive definite (SPD) under the following conditions. The feature array is fixed, and every retained neighbor pair also occurs in the reverse direction. The matrix Ω of nonnegative kernel weights is then symmetric. With positive row sums $D_i = \sum_j \Omega_{ij}$ and ideal normalization $S = D^{-1/2}\Omega D^{-1/2}$, the matrix S is symmetric and has eigenvalues in $[-1, 1]$. Therefore,

$$\star = (1-\beta)I + \beta S, \qquad \lambda(\star) \subseteq [1-2\beta, 1], \qquad 0 < \beta < \tfrac{1}{2}, \tag{S10}$$

is SPD. This bound uses the ideal normalization. The implementation adds a small constant to the denominator. After all truncation steps, the retained lists must be checked for neighbor pairs missing their reverse. The configured procedure does not ensure that both directions are retained (section S4). The identity term alone does not make an asymmetric star symmetric.

If the controller stars $A = \star_0^{-1}$ and $\star_1$, which have separate parameters, are SPD, then $Q = d_0^\top \star_1 d_0$ is symmetric positive semidefinite (PSD), and

$$A^{-1/2} L_v A^{1/2} = A^{1/2} Q A^{1/2} \tag{S11}$$

is symmetric PSD. Thus $L_v = AQ$ has real, nonnegative eigenvalues under these assumptions. The notation $\star_0^{-1}$ names an operator learned separately.

The two terms of the VEF edge operator L_e contain stars with separate parameters. Individual SPD stars do not ensure that both terms are symmetric and PSD under the same inner product. The proof for the vertex operator therefore does not prove that the summed edge operator has real, nonnegative eigenvalues. The complete controller block also contains value and output projections, residual connections, and nonlinear transformations. It is not a PSD linear operator.

S3.3 Expert Fourier Bandwidths

Each expert receives Fourier features of the standardized vertex coordinates $\mathrm{pos} \in \mathbb{R}^{N\times 2}$. For expert $k = 1, \ldots, K$, the frequency matrix is $B_k = \sigma_k G_k \in \mathbb{R}^{2\times n_F}$, where $n_F = 8$ and the entries of G_k are sampled independently from a standard normal distribution. The matrices are sampled at initialization and remain fixed during training. The configured scales are

$$(\sigma_1, \ldots, \sigma_K) = (1.0, 1.7, 2.8, 4.8, 8.0, 13.5, 22.6, 38.0). \tag{S12}$$

The product $\mathrm{pos}\, B_k \in \mathbb{R}^{N\times n_F}$ gives eight coordinate projections per vertex. Applying sine and cosine elementwise produces the Fourier features

$$[\sin(2\pi\, \mathrm{pos}\, B_k)\,;\, \cos(2\pi\, \mathrm{pos}\, B_k)] \in \mathbb{R}^{N\times 2n_F}. \tag{S13}$$

The semicolon joins arrays along the feature dimension. Larger σ_k increase the sampled frequencies, giving the experts different positional inputs. These scales do not restrict an expert to a particular frequency band. The learned atom assignment determines the auxiliary targets and need not follow the order of the Fourier scales. The experts use the Fourier features as inputs. The analyzer constructs their targets separately, using the smoothing operations and differences between smoothed fields described in section 3.

S3.4 Expert Input Construction

The projections include bias terms, which are omitted from the equations. The $2n_F = 16$ Fourier features in eq. (S13) are joined with the shared embedding h. A projection maps the combined features to width $d_x = 88$:

$$z_k^0 = W_{\mathrm{in}}^k[h\,;\, \sin(2\pi\, \mathrm{pos}\, B_k)\,;\, \cos(2\pi\, \mathrm{pos}\, B_k)]. \tag{S14}$$

S3.5 Transformer experts and mixture

Each expert processes the input z_k^0 defined in section S3.4.

Each expert contains $L_x = 2$ transformer blocks. Layer normalization is applied before attention and before the feedforward network. Attention uses softmax over all vertices, with $h_x = 8$ heads of width $d_x/h_x = 11$. The feedforward width is $4d_x = 352$. The experts do not use incidence matrices or attention masks. All eight experts are evaluated for every vertex.

Each standard transformer block applies multi-head softmax self-attention, then a feedforward network separately to each token. Layer normalization is applied before each operation, and a residual connection adds each sublayer's input to its output. For a token array $X \in \mathbb{R}^{n\times D}$ and J heads of width $q = D/J$, the attention sublayer computes

$$\begin{aligned}
&\bar{X} = \mathrm{LN}_1(X), \qquad Q_a = \mathrm{Linear}_{Q,a}(\bar{X}), \quad K_a = \mathrm{Linear}_{K,a}(\bar{X}), \quad V_a = \mathrm{Linear}_{V,a}(\bar{X}),\\
&A_a = \mathrm{softmax}_{\mathrm{row}}\left(\frac{Q_a K_a^\top}{\sqrt{q}}\right), \qquad O_a = \mathrm{Drop}(A_a)V_a,\\
&\widetilde{X} = X + \mathrm{Drop}(\mathrm{Linear}_O[O_1; \ldots; O_J])\,.
\end{aligned} \tag{S15}$$

The row-wise softmax assigns one distribution over all tokens to each query. The feed-forward sublayer then gives

$$\begin{aligned}
X^+ &= \widetilde{X} + \mathrm{Drop}\Big(\mathrm{FFN}(\mathrm{LN}_2(\widetilde{X}))\Big)\,,\\
\mathrm{FFN}(x) &= \mathrm{Linear}_{4D\to D}(\mathrm{GELU}(\mathrm{Linear}_{D\to 4D}(x)))\,.
\end{aligned} \tag{S16}$$

All projections include bias terms, and each block has its own parameters. Every token can attend to every other token. The feedforward network is applied separately to each token. Dropout on attention weights and residual updates uses probability 0.1.

After final layer normalization, $z_k = \mathrm{LN}(z_k^{L_x}) \in \mathbb{R}^{N\times d_x}$. Each expert has two output heads, producing $\mathrm{proj}_k(z_k) \in \mathbb{R}^{N\times d}$ and the band prediction $\hat{b}_k \in \mathbb{R}^{N\times 4}$. Routing and mixing are

$$g_k(v) = \frac{\exp(\ell_k(v))}{\sum_{k'=1}^{K} \exp(\ell_{k'}(v))}, \qquad h_{\text{mix}}(v) = \sum_{k=1}^{K} g_k(v)\, \text{proj}_k(z_k)(v). \tag{S17}$$

The mixture module applies dropout to the combined representation. The model then applies layer normalization and a second dropout before decoding. The dropout probability is 0.1. There is no residual connection around the mixture.

Auxiliary band predictions are trained against the assigned targets in eq. (15). These targets sum to the reference field by construction. The final prediction is decoded from the combined expert features, so it need not equal the sum of the auxiliary predictions.

S3.6 Field decoder

The decoder creates four field tokens per vertex by adding learned embeddings $E_{\text{field}} \in \mathbb{R}^{4\times d}$ to the combined expert features after layer normalization and dropout. Two transformer blocks attend across the four fields at each vertex, with width 192, four heads, and feedforward width 768. They do not attend across vertices. The shared output network, **LayerNorm** $\rightarrow$ **Linear(192,64)** $\rightarrow$ **GELU** $\rightarrow$ **Linear(64,1)**, returns standardized field predictions $\hat{u} \in \mathbb{R}^{N\times 4}$. The implementation includes the batch dimension, giving shape $(B_{\text{batch}}, N, 4)$.

Let $m(v)$ denote the combined expert features after normalization and dropout. The decoder input tokens are

$$t^0_{vc} = m(v) + E_{\text{field},c}, \qquad T^0_v = [t^0_{v1}; \ldots; t^0_{v4}] \in \mathbb{R}^{4\times 192}. \tag{S18}$$

At each vertex, the decoder applies two blocks with separate parameters, using eqs. (S15) and (S16) with $n = 4$, $D = 192$, and $J = 4$. Each attention head has width 48. Attention combines information across the output fields at that vertex. The output network maps the final token for channel c to

$$\hat{u}_{vc} = \text{Linear}_{64\to 1}\big(\text{GELU}\big(\text{Linear}_{192\to 64}(\text{LN}(t^2_{vc}))\big)\big). \tag{S19}$$

The same readout parameters are used for all vertices and fields.

S3.7 Initialization and parameter counts

After the model is created, Xavier-uniform initialization is applied to every parameter tensor with more than one dimension, including field embeddings and assignment logits. The fixed Fourier matrices are stored separately from trainable parameters and are unchanged by this step. Evaluation logs report 3,106,490 parameters marked as trainable for the proposed VEF configuration and 3,023,962 for the vertex-only comparison. These totals include unused parameters and analyzer parameters that receive no gradients from the training loss.

S3.8 Analyzer and assignment gradients

The analyzer uses a separate diagonal-star vertex operator to construct nine target components, called atoms, from smoothed reference fields and their differences. An edge-feature network computes the conductances. Before multiplication by the learned assignment matrix, the atoms are treated as constants when computing gradients. The auxiliary band loss and the penalty for assigned-target shares below the threshold therefore update the assignment logits without updating the conductance network. The conductance parameters remain at initialization. The analyzer is used only during training. The auxiliary expert predictions are not inputs to the final decoder.

Each column of the assignment sums to one, so the assigned targets sum to the reference field by construction. This identity does not require orthogonal atoms or a fixed expert order. The final Xavier initialization replaces the assignment values set by the constructor. The Fourier scales remain fixed and do not require the assignment to follow their order.

S4 Experimental Protocol

This section gives the implementation and training settings for the proposed VEF configuration and the vertex-only comparison (V). Tables S2 and S3 list the training and architecture settings. Lift

is computed using the same contour on the working mesh for every model. The separate contour comparison in section S7 uses a subset of cases and the vertex-only checkpoint.

Table S2: Training configuration.

optimiser	AdamW [41], weight decay 10^{-4}
initial stage	400 epochs; per-batch OneCycleLR [42], peak learning rate 10^{-3}, warm-up fraction 0.3, division factor 25, final division factor 10^4; initial rate 4×10^{-5} and final rate approximately 4×10^{-9}
additional VEF training	resume from epoch 400 for 200 more epochs; per-batch OneCycleLR, peak learning rate 2×10^{-4}, warm-up fraction 0.1, division factors 25 and 10^4
batch	one whole mesh
precision	automatic mixed precision with `bfloat16`; incidence-matrix products and all analyzer operations computed in 32-bit floating point (`fp32`)
dropout	0.1 throughout
initialisation	Xavier uniform on every parameter of dimension greater than one
checkpoints	every 20 epochs; VEF: 30 checkpoints, selected epoch 560; V: 20 checkpoints, selected epoch 380
logging	per-epoch metrics in `metrics.jsonl`
hardware	one NVIDIA RTX 4090 with 24 GB memory

Table S3: Architecture. Settings of the proposed VEF configuration and the vertex-only comparison. Dropout is 0.1 throughout; parameter tensors of dimension greater than one are Xavier-uniform initialized.

part	symbol	value
inputs	(c_v, c_e, c_f)	$(9, 21, 13)$ channels on vertices, edges and faces
embedding	d	192; $W_1 : 9 \to 192$, GELU, $W_2 : 192 \to 192$ (eq. (4))
experts	K	8
	d_x	88, with $W_{\text{in}}^k : (d + 2n_F) = 208 \to 88$ and $\text{proj}_k : 88 \to 192$
	L_x, h_x	2 blocks with normalization before attention and feedforward updates; 8 heads of dimension 11; feedforward width $4d_x = 352$
	n_F, σ	8 frequencies giving 16 sine/cosine channels; $\sigma = (1.0, 1.7, 2.8, 4.8, 8.0, 13.5, 22.6, 38.0)$
controller	d_c, L_c	64, 2 Gaussian-Hodge layers
	H	4 heads, feed-forward $4d_c = 256$, read-out $64 \to 64 \to K$
	$\mathcal{N}$	VEF: vertex/edge/face caps $64/32/1$; V: vertex/edge caps $64/64$
analyzer	hidden	64
	smoothing	$(2, 4, 6, 8, 12, 16, 24, 32)$ steps, giving 9 atoms
	$\star_0$	the sum of incident edge conductances at each vertex, rather than a separately learned vertex weight; the resulting operator has $\lambda_{\max} \leq 2$
decoder	blocks	2 blocks with normalization before attention and feedforward updates; 4 heads attending across the four output-field tokens at each vertex; feedforward width 768
	read-out	$E_{\text{field}} \in \mathbb{R}^{4 \times 192}$ and a shared MLP $: 192 \to 64 \to 1$
parameters	total	3,106,490 (VEF); 3,023,962 (V)

Input arrays and normalization

The input arrays have 9, 21, and 13 channels on vertices, edges, and faces. Vertex inputs are the two physical position coordinates, barycentric dual area, two inlet velocity components, signed distance, two supplied normal components, and the wall indicator. All nine columns are standardized with fixed training statistics. The standardized position columns are also used for the expert Fourier features.

Edge inputs concatenate twelve coordinate entries for the edge endpoints and the opposite vertices of the incident triangles, three components of the normalized mean endpoint normal, and six pairwise distances among those four vertices. Missing opposite-vertex coordinates at a boundary are zero-filled. Face inputs concatenate the nine ordered triangle-coordinate entries, three face-normal components, and the triangle area. Geometry is computed with coordinates embedded at $z = 0$. Edge and face arrays are supplied without dataset-level standardization. Later embedding and layer-normalization operations are separate transformations. The term raw edge or face features refers to these unstandardized input arrays.

The target scaling is given in eq. (3): velocity and turbulent kinematic viscosity are divided by the sample's freestream speed, and pressure by its square. A fixed mean and standard deviation are then applied to each output channel. The statistics are computed over vertices from the selected training

cases' fixed `s0` meshes, with a configured limit of 100 cases. Each vertex has equal weight. These per-channel statistics are reused for all training and test cases. The additional VEF training uses the saved statistics from the initial stage. Evaluation reverses standardization and freestream scaling for metrics in physical units. The turbulent-viscosity target remains linear, and its division by speed leaves units of length.

Training cases and checkpoint selection

The Hodge-routed configurations train on all 800 official training cases. Each time a training case is loaded, one of its available remeshes is selected, from a pool of up to eight. Evaluation uses a fixed mesh for each of the 200 official test cases. The external baselines reserve 80 of the 800 training cases for validation and train on the remaining 720. Their checkpoints are selected using validation errors.

The reported Hodge-routed checkpoints are selected by mean relative L_2 error on the 200 test cases: epoch 560 among 30 checkpoints for VEF, and epoch 380 among 20 checkpoints for V. The reported test errors are therefore also used to choose the checkpoint. Error differences from the external baselines cannot be attributed to architecture alone because the training cases and checkpoint-selection procedures differ. The external all-input configuration has eight channels and omits the wall-indicator channel used by the Hodge-routed configurations (section S6).

Neighbor lists

Neighbor lists are computed once per mesh and remain fixed during training and evaluation. The stored vertex and edge lists contain at most $m = 256$ entries and use graph distances up to $R = 8$ steps. For a vertex list, ring r contains vertices r steps away in the vertex-adjacency graph. For an edge list, it contains edges r steps away in the edge-adjacency graph. The stored list for element i is $\mathcal{N}^{\mathrm{full}}(i)$.

Ring 0 contains the element itself, $\{i\}$. The remaining places are divided among rings 1 to R in proportion to δ^{r-1}, with $\delta = 0.55$ and at least one place per ring. The rounded limits are $(116, 64, 35, 19, 11, 6, 3, 2)$. These limits and the self entry can exceed the total cap by one. If too many entries are selected, the sampler keeps the first m in order of increasing graph distance. If a ring contains no more neighbors than allowed, all are kept. Otherwise, the allowed number is sampled uniformly at random. Any remaining places are filled with the nearest unselected elements in graph distance. For every selected neighbor pair, its reverse is then added:

$$\mathcal{N}^{\mathrm{full}}(i) \;\leftarrow\; \mathcal{N}^{\mathrm{full}}(i) \cup \{\, j : i \in \mathcal{N}^{\mathrm{full}}(j) \,\}, \tag{S20}$$

If adding reverse pairs makes a list longer than m, it is shortened to m entries. Each face list contains only the face itself, with $m = 1$. Under ideal normalization, a Gaussian star using only self neighbors is the identity. The implementation adds a small constant to the denominator. The VEF face star uses the learned positive diagonal weights in eq. (7).

The random seed used to sample each mesh's lists is computed from its name. The lists are saved and reused between epochs. After adding reverse pairs as in eq. (S20), the implementation places the element itself first, sorts the other entries by element index, and keeps at most m entries. This truncation can remove one direction of a neighbor pair.

The controller uses the first entries of these stored lists: 64 vertex entries and 32 edge entries for VEF, compared with 64 vertex and 64 edge entries for V. Each face list contains only itself. Unused positions contain -1 and are excluded from the operator. Because entries are sorted by element index, the selected entries need not be the nearest neighbors or contain complete graph-distance rings. Selecting the first entries can also remove one direction of a neighbor pair. The procedure therefore does not ensure that every retained pair occurs in both directions. The eigenvalue bounds in section S3.2 require both directions after all these steps.

Saved geometry files are identified by mesh name. Their names do not record the neighbor-list settings. Reusing a saved file does not rebuild its lists with the current settings. When checking or changing neighbor-list settings, the saved lists must also be checked.

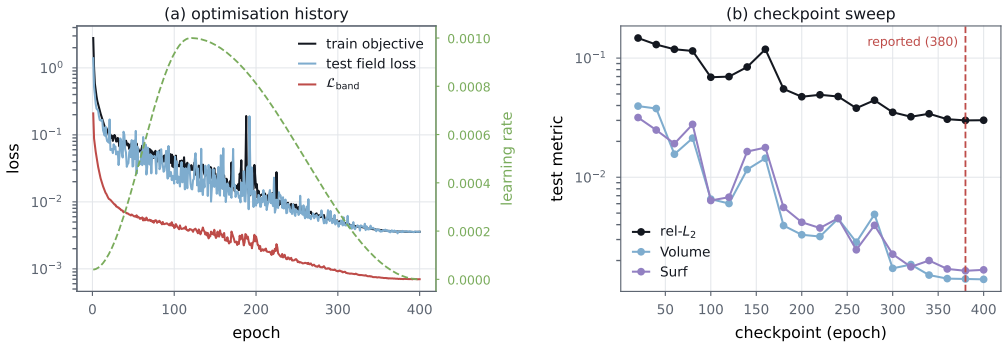


Figure S1: Training the vertex-only comparison. (a) Training objective, test-set field loss, and auxiliary band loss, with the learning rate on the right axis. The training objective includes auxiliary terms absent from the test-set field loss. (b) The 20-checkpoint test sweep, with epoch 380 selected by mean relative L_2 error. These curves do not describe VEF training.

Vertex-only training curves

Figure S1 shows the training curves for the vertex-only comparison. The learning rate reaches 10^{-3} at epoch 120 and decays to approximately 4×10^{-9} by epoch 400. The training curve includes auxiliary loss terms. The test-set field-loss curve excludes these terms and is computed on the official test loader, although its original logging key is `val_loss`. The auxiliary band loss decreases approximately 300-fold. Checkpoint evaluation gives vertex-only relative L_2 errors of 0.0307, 0.0300, and 0.0301 at epochs 360, 380, and 400, with corresponding Volume errors 0.00141, 0.00140, and 0.00139. The VEF training settings and selected checkpoint are listed separately in tables S2 and S10.

S5 The Model Family

Table S4 describes the proposed VEF configuration and the comparison models. Models are grouped by how mesh structure enters field prediction or auxiliary supervision. Table S6 reports the evaluated comparisons.

Mesh-free predictors. The internal plain-transformer variants read the nine per-vertex channels of section 3.3. The *plain transformer* uses four pre-normalized softmax transformer layers of width 64 with global vertex attention, without incidence operators or neighbor supports. The *mesh-windowed* variant restricts attention to the geodesic-ring support, and the *locality-biased* variant adds a learned distance bias to global attention logits. The *wide plain transformer* uses width 192. The + *lift loss* variant adds a differentiable lift term to its objective.

Frequency mixtures of experts with a feature-space router. These models combine transformer experts using per-vertex weights from an MLP on the shared vertex latent. Mesh structure enters the auxiliary target decomposition, without incidence operators in the prediction path. The *fixed-band MoE* uses four experts and a fixed adjacency ladder $(2, 8, 32)$ with no learned operator: the target is low-passed by powers of the symmetric normalised adjacency $D^{-1/2}(A+I)D^{-1/2}$, with A the vertex adjacency of the mesh graph and D its degree matrix, and the four bands are the successive differences of those low-passes, telescoping to the target as the atoms of eq. (14) do. This operator is not the analyzer of section 3.7 with its conductances held fixed: it uses a different normalization and no Euler step size. The two *entropy-balanced* variants use a parameterized Hodge analyzer and maximize the entropy of assigned-target squared-norm shares, with differentiable or detached atom construction, respectively. The configuration named *learned-band MoE* uses eight experts, the diagonal-star analyzer, the learned assignment of eq. (15), and a one-sided norm-floor penalty. Its atoms are detached, so learning the assignment does not optimize the analyzer conductances (section S3.8). The name denotes learned target assignment rather than prescribed frequency boundaries. The + *wall-surface branch* adds true-surface input channels, a one-dimensional boundary transformer along the wall, and a lift-weighted surface loss. The fixed-band and learned-band configurations also differ in expert count, so their comparison does not isolate the target construction alone.

Hodge-routed mixtures. The proposed *Hodge-routed MoE (VEF)* computes gate logits with two Gaussian-Hodge layers that update vertex, edge, and face streams, followed by a vertex readout. Its prediction experts, target assignment, and field decoder follow the learned-band construction. The *vertex-only* comparison (V) uses the vertex Hodge update and retains position-wise edge and face feedforward updates. Its *wide-expert* configuration increases the total model size to 5.68M, with shared width 256 and expert width 128. VEF and V differ in edge support, training duration, and checkpoint count as well as active Hodge updates (section S8); their comparison does not isolate cochain degree alone.

Hodge predictors. Unlike the controller of section 3.5, this line keeps the learned Hodge operator in the *prediction* path, as in HodgeFormer [21]. The *PSD-star HodgeFormer* learns Gaussian stars, symmetric positive definite under a mutual support (section 3.5.1), and assembles L_0 from them, with per-field MLP read-out heads. Adding a cross-field attention decoder, two global blocks and per-case normalisation gives the $+$ *cross-field decoder* variant, and replacing its loss by the disjoint volume/surface field terms of eq. (17) gives the $+$ *disjoint surface loss* variant, which is the member with the lowest mean relative L_2 error among the three Hodge predictor rows in table S6. Additional configurations change support symmetry, the vertex star, the bound on β, the kernel, or layer count.

External baselines. Transolver [17], PointNet [43] and a pointwise MLP are trained natively on the working meshes and evaluated using the same metric definitions. Training-set allocations and checkpoint selection differ as stated in section 4.1. Their input arms are named by which channels they receive, where *all inputs* is position, dual area, inlet velocity, signed distance and wall normals; *pos+inlet* is the minimal arm; and *+SDF+normals* omits the dual area. These baseline arms have eight channels for all inputs, four for position plus inlet, and seven with SDF and normals; they do not include the ninth wall-indicator channel supplied to our predictor.

Table S4: The model family. Every trained variant in this paper, grouped by how mesh structure reaches the prediction. *Router* means the operator produces only gate logits; *bands* means the mesh enters only through the supervisory decomposition of the target; *operator* means the mesh is in the prediction path; *none* means a point-set model.

name used in this paper	what it is	params
mesh-free predictors, where mesh use is none		
Plain transformer	4-layer pre-norm softmax transformer, width 64, global attention over all vertices; the Hodge operator deleted	0.31M
windowed	attention restricted to the geodesic-ring support used by the operator	0.31M
+ locality bias	global attention + a learned soft distance bias on the logits	0.31M
Wide plain transformer	the same at width 192 (FFN 768, 12 heads); the width ablation	2.72M
+ lift loss	+ a differentiable C_L contour integral in the objective	2.72M
frequency MoE, feature-space (MLP) router, where mesh use is bands		
Fixed-band MoE	4 band experts, per-vertex gate, fixed adjacency ladder $(2, 8, 32)$	2.75M
entropy-balanced	parameterized Hodge analyzer with differentiable atoms and an entropy objective	2.75M
entropy-balanced, detached	the same with detached atoms	2.75M
Learned-band MoE	8 experts, detached analyzer atoms, learned assignment, one-sided norm floor	2.75M
+ wall-surface branch	+ true-surface channels, a 1-D wall transformer and a lift-weighted surface loss	3.00M
Hodge-routed MoE, where mesh use is router		
Hodge-routed MoE (VEF)	proposed model; vertex, edge, and face Hodge updates; vertex readout	3.11M
Vertex-only controller (V)	vertex Hodge update; position-wise edge and face feedforward updates	3.02M
wide experts (V)	vertex-only controller; shared width 256, expert width 128	5.68M
Hodge predictors, where mesh use is operator		
PSD-star HodgeFormer	Gaussian stars, L_0 in the prediction path, per-field heads	0.45M
+ cross-field decoder	+ cross-field attention decoder, 2 global blocks, per-case normalisation	0.47M
+ disjoint surface loss	+ the disjoint volume/wall MSE of eq. (16) and a wall tag; best of the family	0.47M
symmetric support	mutual effective support; conditional vertex spectrum in section S3.2	0.47M
no vertex star	$\star_0^{-1}$ dropped, operator $= d_0^\top \star_1 d_0$	0.47M
unbound β	the $\beta < \frac{1}{2}$ cap released (β climbs to 0.86)	0.47M
dot-product kernel	the RBF diffusion kernel replaced by attention on the same support	0.47M
1 / 4 layers	the depth sweep of the same operator	0.35/0.72M
external baselines, where mesh use is none		
Transolver [17]	physics-slice attention; arms differ only in input channels	6.00M
PointNet [43]	shared per-point MLP with permutation-invariant pooling	0.08M
Pointwise MLP	per-vertex MLP, no interaction between vertices, all inputs	0.02M

S6 Input-Feature Ablation of the Baseline

The Transolver baseline was retrained on the working mesh at four input arms, so that its row in the main comparison can be read against the features it was given. The arms differ only in the per-vertex channels, where *pi* is position and inlet velocity, *pia* adds the dual-cell area, *pisn* adds the signed distance and the wall normals instead, and *pisna* is all of them, which is the arm reported in the main paper. All four use the same architecture, the same split and the same metric code.

Table S5: Transolver by input arm. Metrics on the working meshes for the official $800/200$ split. The training allocation and validation-based checkpoint selection are specified in section S4. Best values per metric are in bold.

arm	channels	params	rel-L_2 ↓	R^2 ↑	Volume↓	Surf↓	C_L ↓	ρ_L ↑
`pi`	pos + inlet	5,995,332	0.0439	**0.9954**	0.00270	0.00140	**0.0078**	0.99988
`pia`	+ area	5,995,844	0.0592	0.9922	0.00418	0.00282	0.0115	0.99980
`pisn`	+ SDF + normals	5,996,868	0.0440	0.9952	0.00286	**0.00137**	0.0098	0.99977
`pisna`	+ SDF + normals + area	5,997,380	**0.0436**	0.9953	**0.00265**	0.00143	0.0103	**0.99989**

The spread across arms on the field metrics is 0.0436 to 0.0592; the highest relative L_2 is `pia`, which adds the dual area without the signed distance and the wall normals. The all-input arm has the lowest relative L_2 and Volume errors. The position-plus-inlet arm has the highest R^2 and lowest lift error, and the SDF-plus-normals arm has the lowest Surf error. The four arms contain 4, 5, 7, and 8 channels, respectively; none includes the wall-indicator input used by our predictor.

S7 Complete Results

Table S6 reports the proposed VEF configuration and the evaluated comparison models described in table S4. Table S7 gives the per-field breakdown for selected models, table S8 decomposes the two AirFRANS-native metrics per field, table S9 reports the checkpoint sweep, and fig. 3 shows one test case from the vertex-only checkpoint at epoch 380. This qualitative example and the diagnostics in section S9 describe the vertex-only comparison.

The VEF results correspond to case `v57_gated_hodge_vef` at epoch 560; the vertex-only results correspond to `v57_gated_hodge` at epoch 380. Their field and Volume/Surf values were checked against the saved `test_metrics.npz` files, and their checkpoint counts against the sweep summaries. Complete-comparison rows report all metrics at the checkpoint selected by mean relative L_2. The sweep table separately lists the minimum of each stated metric and its epoch.

Table S6: Complete comparison. Official AirFRANS $800/200$ split, own-grid evaluation on the $\approx$4.9k-vertex mesh, the same test meshes and evaluation metrics for every row. VEF is the proposed configuration; V is the vertex-only comparison. Objectives differ across loss ablations; training and selection protocols are specified in section S4. Models are described in section S5. Best per column in **bold**.

model	mesh use	params	rel-L_2 ↓	R^2 ↑	Volume↓	Surf↓	C_L ↓
Hodge-routed MoE (VEF)	router	3.11M	0.0359	0.9978	0.00182	0.00180	0.0277
Hodge-routed MoE (V)	router	3.02M	**0.0300**	**0.9983**	**0.00140**	0.00164	0.0225
wide experts (V)	router	5.68M	0.0341	0.9979	0.00163	0.00219	0.0286
Learned-band MoE	bands	2.75M	0.0345	0.9979	0.00165	0.00192	0.0265
Fixed-band MoE	bands	2.75M	0.0351	0.9978	0.00178	0.00184	0.0280
+ wall-surface branch	bands	3.00M	0.0378	0.9973	0.00196	0.00227	0.0247
entropy-balanced, detached	bands	2.75M	0.0402	0.9973	0.00206	0.00214	0.0270
entropy-balanced	bands	2.75M	0.0412	0.9972	0.00198	0.00233	0.0322
Transolver, all inputs [17]	none	6.0M	0.0436	0.9953	0.00265	0.00143	0.0103
Transolver, pos+inlet	none	6.00M	0.0439	0.9954	0.00270	0.00140	**0.0078**
Transolver, +SDF+normals	none	6.00M	0.0440	0.9952	0.00286	**0.00137**	0.0098
Wide plain transformer	none	2.72M	0.0446	0.9960	0.00265	0.00192	0.0245
+ lift loss	none	2.72M	0.0458	0.9960	0.00248	0.00296	0.0378
Plain transformer	none	0.31M	0.0544	0.9943	0.00374	0.00300	0.0173
PointNet, all inputs [43]	none	0.08M	0.0689	0.9904	0.00451	0.00440	0.0149
Pointwise MLP, all inputs	none	0.02M	0.1743	0.9404	0.03576	0.03969	0.0860
PSD-star HodgeFormer, + surface loss	operator	0.47M	0.0800	0.9885	0.01028	0.01048	0.0681
PSD-star HodgeFormer, + decoder	operator	0.47M	0.0822	0.9884	0.00803	0.01205	0.0567
PSD-star HodgeFormer	operator	0.45M	0.1154	0.9769	0.01446	0.03174	0.1747

Table S7: Per-field relative L_2 for selected models. Errors for the proposed VEF configuration, the vertex-only configuration (V), and comparison models. Metrics use the selected checkpoint of each run. Best values per field are in bold.

model	u_x	u_y	p	ν_t
Hodge-routed MoE (VEF)	0.0067	0.0348	0.0380	0.0642
Hodge-routed MoE (V)	**0.0056**	0.0259	**0.0255**	**0.0629**
wide experts (V)	0.0072	0.0319	0.0337	0.0638
Learned-band MoE	0.0068	0.0288	0.0365	0.0658
Fixed-band MoE	0.0068	0.0312	0.0346	0.0679
+ wall-surface branch	0.0064	0.0293	0.0342	0.0812
Transolver, all inputs	0.0067	**0.0220**	0.0298	0.1158
Wide plain transformer	0.0059	0.0330	0.0326	0.1068
Plain transformer	0.0067	0.0554	0.0349	0.1205
PSD-star HodgeFormer, best	0.0149	0.0650	0.0852	0.1548
PointNet, all inputs	0.0125	0.0540	0.0655	0.1435
Pointwise MLP, all inputs	0.0518	0.1865	0.1765	0.2825

For VEF, both minimum mean relative L_2 and minimum Volume occur at epoch 560. For the vertex-only comparison, they occur at epochs 380 and 400, respectively. The vertex-only Volume value in the complete comparison is 0.00140 at the relative-L_2-selected checkpoint, rather than the separate minimum 0.00139.

Table S8: Per-field Volume and Surf. AirFRANS-native normalized mean squared errors on non-wall and wall vertices, respectively, at the checkpoint selected by mean relative L_2 for each run. VEF and the vertex-only comparison (V) are listed separately.

	Volume (interior)				Surf (wall)			
model	u_x	u_y	p	ν_t	u_x	u_y	p	ν_t
Hodge-routed MoE (VEF)	0.00019	0.00096	0.00077	0.00534	0.00079	0.00197	0.00272	0.00173
Hodge-routed MoE (V)	0.00011	0.00052	0.00031	0.00466	0.00073	0.00174	0.00248	0.00160
Learned-band MoE	0.00019	0.00063	0.00072	0.00506	0.00087	0.00195	0.00307	0.00181
Fixed-band MoE	0.00019	0.00086	0.00069	0.00539	0.00090	0.00198	0.00296	0.00152
Wide plain transformer	0.00017	0.00109	0.00074	0.00859	0.00062	0.00191	0.00313	0.00201
Plain transformer	0.00019	0.00279	0.00060	0.01140	0.00088	0.00403	0.00431	0.00278
PSD-star HodgeFormer, best	0.00114	0.00476	0.00420	0.03103	0.00406	0.00609	0.01801	0.01378

Table S9: Checkpoint sweep. Separate minimum mean relative L_2 and Volume values with their epochs. † the VEF configuration was trained for 400 epochs plus a 200-epoch continuation with peak learning rate 2×10^{-4} and swept at 30 checkpoints rather than 20.

model	checkpoints	best rel-L_2 (epoch)	best Volume (epoch)
Hodge-routed MoE (VEF)†	30	0.0359 (560)	0.00182 (560)
Hodge-routed MoE (V)	20	0.0300 (380)	0.00139 (400)
wide experts (V)	20	0.0341 (400)	0.00163 (400)
Learned-band MoE	20	0.0345 (380)	0.00165 (380)
Fixed-band MoE	20	0.0351 (400)	0.00178 (380)
Plain transformer	20	0.0544 (320)	0.00339 (400)
PSD-star HodgeFormer, best	20	0.0800 (320)	0.00651 (400)

The VEF mesh-native lift error is $C_L = 0.0277$ over 200 test cases. The vertex-only checkpoint gives $C_L = 0.0225$ and $\rho_L = 0.99988$. A separate 10-case contour comparison using the vertex-only checkpoint gives relative lift errors of 0.0220 for the mesh-native band and 0.0205 for the surface loop. These subset values do not establish a full-test-set comparison of the contour conventions or a VEF contour comparison.

S8 Comparison of Controller Configurations

The proposed VEF controller applies L_0 to vertices, $L_1 = L_1^{\text{down}} + L_1^{\text{up}}$ to edges, and L_2^{down} to faces, using the incidence composition in eq. (8). All Hodge attention updates read the pre-update streams and are computed simultaneously. The vertex-only configuration (V) applies the vertex Hodge attention update without edge or face Hodge attention updates. Its edge and face streams retain position-wise residual feedforward updates; edge features therefore continue to enter the vertex star. Both configurations produce routing logits from the final vertex stream.

The VEF run uses an edge support cap of 32, compared with 64 for V. It also includes a 200-epoch continuation and evaluates 30 checkpoints rather than 20. The comparison therefore does not isolate cochain degree alone. Table S10 reports both selected runs.

Face supports contain one entry. Under ideal normalization, the self-only Gaussian star is the identity; the implemented denominator offset gives an approximately identity operator. VEF uses the learned positive diagonal face star of eq. (7), with $z_i = \|\varphi_i\|^2/(d_h\sigma^2)$. This scaling is not constrained to equal a geometric face-area star. Incidence applications flatten head and channel axes before sparse multiplication, avoiding a separate sparse call for each head.

The selected VEF run has higher errors than the selected vertex-only run on the metrics in table S10. Differences in support, training duration, and checkpoint count, together with one run per configuration, limit attribution of the change to the active Hodge updates. Among the configurations in table S6, the vertex-only Hodge-routed MoE has the lowest mean relative L_2 error; other models have lower surface and lift errors.

Table S10: VEF and vertex-only (V) controllers. VEF is the proposed configuration. Training duration, support caps, and checkpoint counts differ between the runs. All errors are evaluated at the checkpoint selected by mean relative L_2 over 200 test cases. The final column gives VEF relative to V; percentage changes use unrounded metrics. Bold values are the better of the two configurations for each metric.

	VEF (proposed)	V (vertex-only)	VEF relative to V
operators	L_0, $L_1^{\text{down}}+L_1^{\text{up}}$, L_2^{down}	L_0	
supports	vertex/edge/face $64/32/1$	vertex/edge $64/64$	
params	3,106,490	3,023,962	+82,528 (+2.7%)
epochs / checkpoints	$400+200$ / 30	400 / 20	
selected checkpoint	560	380	
rel-$L_2\downarrow$	0.0359	**0.0300**	+19.7%
$R^2\uparrow$	0.9978	**0.9983**	
Volume↓	0.00182	**0.00140**	+29.7%
Surf↓	0.00180	**0.00164**	+10.1%
C_L (band)↓	0.0277	**0.0225**	+23.5%
u_x	0.0067	**0.0056**	+18.5%
u_y	0.0348	**0.0259**	+34.1%
p	0.0380	**0.0255**	+48.9%
ν_t	0.0642	**0.0629**	+2.0%

S9 Diagnostics of the Vertex-only Configuration

This section retains the analyzer, assignment, and gate diagnostics for the vertex-only configuration (V) at epoch 380. These figures and measurements do not describe the proposed VEF checkpoint at epoch 560. The analyzer's diagonal stars are

$$\star_1 = \mathrm{diag}(w_e), \quad w_e = \mathrm{softplus}(\mathrm{MLP}_e(x_e)) + \epsilon_w; \qquad \star_0 = \mathrm{diag}(\mu_v), \quad \mu_v = \sum_{e \ni v} w_e, \tag{S21}$$

where w_e, the edge conductance, is the analyzer's only parameterised content. The atoms are the telescoping differences of the forward-Euler smoothing ladder $\bar{y}^{(j)} = (I - \tau L_0)^{m_j} y$,

$$a_0 = \bar{y}^{(L-1)}, \qquad a_i = \bar{y}^{(L-1-i)} - \bar{y}^{(L-i)} \quad (1 \le i \le L-1), \qquad a_{M-1} = y - \bar{y}^{(0)}, \tag{S22}$$

ordered from the most-smoothed component (a_0) to the residual after the fewest smoothing steps (a_{M-1}), and the assignment P groups them into per-expert band targets t_k,

$$P_{km} = \frac{\exp(\theta_{km}/T)}{\sum_{k'} \exp(\theta_{k'm}/T)}, \qquad \sum_k P_{km} = 1 \ \ \forall m, \qquad t_k = \sum_{m=0}^{M-1} P_{km}\, a_m. \tag{S23}$$

The analyzer atoms are detached before assignment, so its conductance network receives no gradient from eq. (16) and remains at initialization (section S3.8). Conductance is a function of edge geometry. Its contribution to smoothing is normalized by the incident degree: the weight of neighbor j at vertex i is proportional to w_{ij}/μ_i. Larger unnormalized conductance alone does not establish faster removal of local field structure. Labels referring to a learned analyzer in the retained image panels identify the parameterized module; its conductances were not optimized by this objective.

Figures S2 and S3 illustrate the pressure decomposition and assignment at the epoch-380 checkpoint. The plotted atom shares are normalized squared norms, $\|a_m\|^2 / \sum_{m'} \|a_{m'}\|^2$, for the illustrated field. The rounded shares are $[0.812, 0.003, 0.006, 0.003, 0.006, 0.003, 0.006, 0.019, 0.143]$. Atoms overlap, so these values are not an orthogonal partition of the target's energy. Approximately 81% of the summed squared atom norms is in the coarsest atom and 14% in the finest residual. These case-level diagnostics do not establish corresponding test-set means.

The energy-floor penalty acts on assigned targets t_k, not raw atoms a_m. Its activation therefore depends on the assigned-target norm shares. The vertex-only training log records a zero floor penalty at epoch 380; this value does not establish inactivity throughout training. The constructor's preferred atom-to-expert grouping is overwritten by global Xavier initialization, so no prescribed ordering is imposed on the learned assignment.

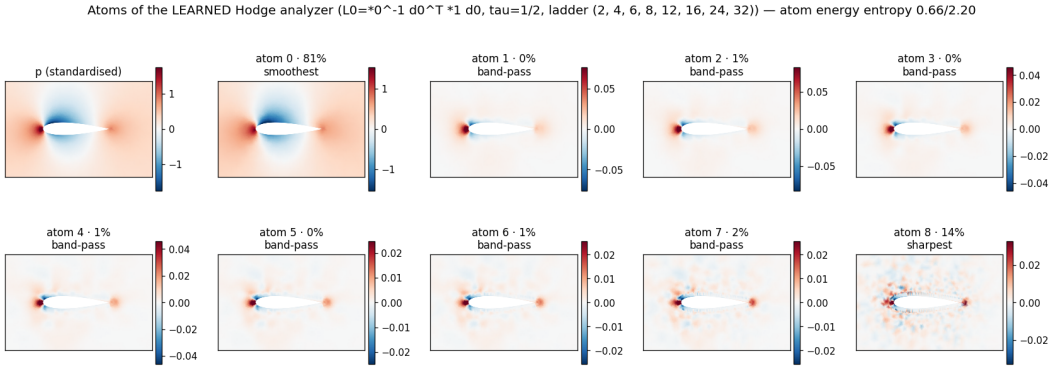


Figure S2: Vertex-only analyzer atoms at epoch 380**.** Nine telescoping atoms for the illustrated pressure field. Atom 0 is the most-smoothed component; atoms 1–7 are differences between smoothing levels, and atom 8 is the residual after the fewest smoothing steps. The first and last components account for approximately 81% and 14% of the summed squared atom norms. Their sum reconstructs the field algebraically, subject to floating-point rounding (eq. (14)). The conductance network remains at initialization.

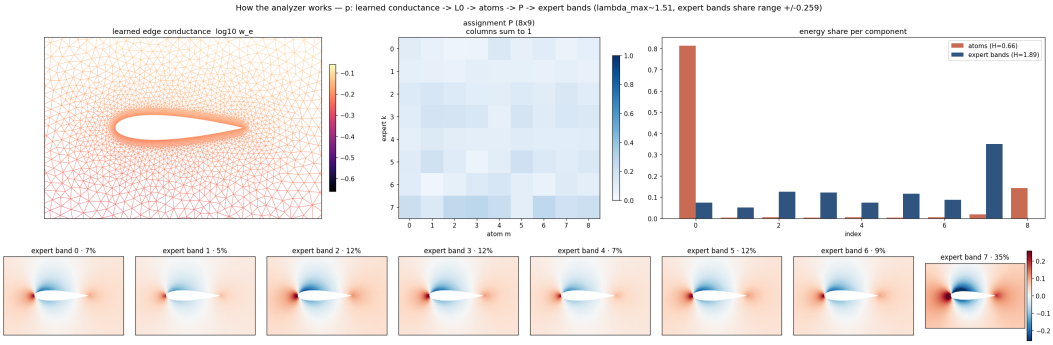


Figure S3: Vertex-only analyzer and assignment at epoch 380**.** Edge conductance w_e from the initialized conductance network; the learned 8×9 assignment P (eq. (15)); the eight assigned targets on a shared symmetric colour range; and normalized squared component norms before and after assignment. Global Xavier initialization overwrites the constructor's preferred assignment logits.

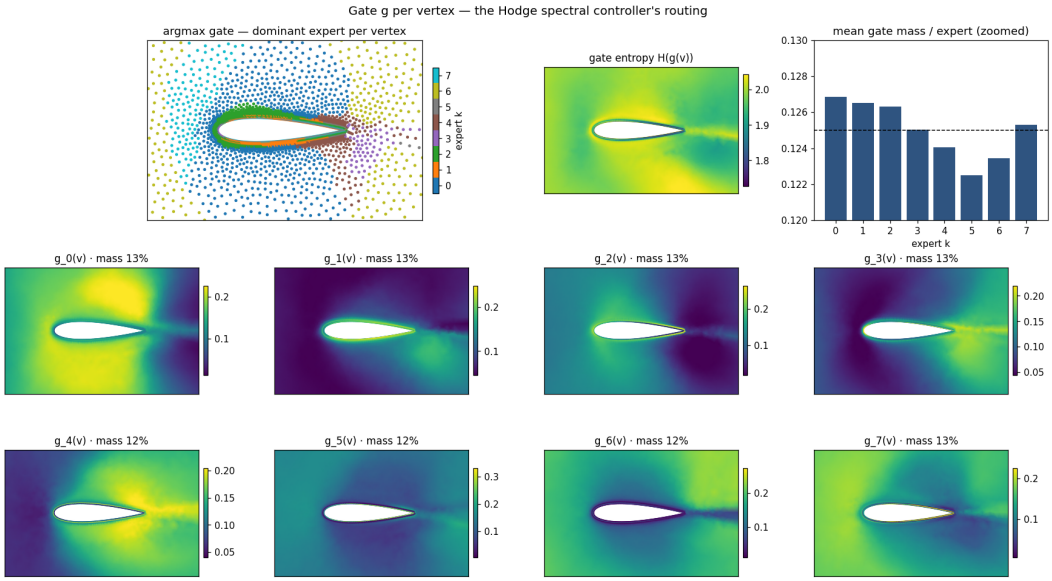


Figure S4: Vertex-only gate $g(v)$ at epoch 380**.** Top: dominant expert at each vertex, per-vertex gate entropy, and mean gate mass per expert against the uniform $1/K$ line (vertical axis restricted to 0.120–0.130). Bottom: the eight weight fields $g_k(v)$. The dominant-expert panel displays an argmax of soft weights; all experts are evaluated. These panels describe routing for the displayed mesh rather than averages over the test set.

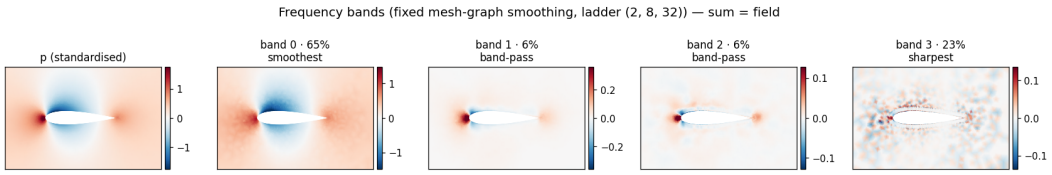


Figure S5: The fixed reference decomposition of the pressure field with the ladder $(2, 8, 32)$ on the unweighted adjacency, for comparison with the vertex-only analyzer example in fig. S2. This decomposition is independent of a trained controller checkpoint. The normalized squared component norms for this illustrated field are $[0.65, 0.06, 0.06, 0.23]$; the components overlap.